Ben Bubnick
Interview STAR Story Bank

Version 1.3 | Updated November 1, 2026 | 127 stories

This is a searchable interview reference. Use the accomplishment as the prompt, then scan the highlighted **S**, **T**, **A**, and **R** rows. Use the role index or PDF search to move directly to the relevant story.

Role Index

- **HHAeXchange – Senior Data Engineer** 13 stories |p. 2
- **GrowData Analytics – Principal ML Engineer** 9 stories |p. 6
- **Adonis – Senior Data Engineer** 8 stories |p. 11
- **Arkos Health – Senior Data Analyst** 20 stories |p. 14
- **Merative – Senior Data Scientist** 7 stories |p. 19
- **IBM – Senior Data Scientist** 13 stories |p. 22
- **IBM – Senior Data Architect** 11 stories |p. 26
- **IBM – Data Architect** 10 stories |p. 30
- **IBM – Software Developer, Data Platform** 6 stories |p. 33
- **IBM – Data Scientist** 5 stories |p. 35
- **Explorys, an IBM Company – Data Scientist** 3 stories |p. 37
- **AmTrust Financial Services – Software Engineer** 9 stories |p. 38
- **PPG Industries – Color Scientist** 13 stories |p. 41

HHAeXchange – Senior Data Engineer

Dates: Oct. 2025 - Jun. 2026
Technologies: Snowflake, dbt, SQL/Jinja, Python, Snowpark, pandas, Liquibase, GitHub Actions, FHIR/Aidbox

Brought in to unify two distinct data environments following the HHAeXchange-Sandata merger. Built shared Snowflake/dbt models and onboarding patterns, then identified and remediated data-quality defects affecting hundreds of millions of EVV records. Led healthcare source analysis, data-quality investigation and remediation, and Snowflake modeling to produce trusted, standardized datasets for reporting and downstream use. Contract via Veersa Technologies.

hhax-01

Engineered unified and analytics Snowflake/dbt models spanning 55 tables across two organizations and two source-system configurations, establishing reusable patterns for faster client onboarding.

S After HHAeXchange and Sandata came together, the reporting layer had two versions of the same healthcare world. Patients, caregivers, offices, and visits existed in both systems, but their keys, types, and lifecycle rules disagreed. Maintaining parallel analytics would have doubled support effort and made every new report a reconciliation exercise.

T I needed to create one dependable analytical contract while keeping the existing HHA reporting path stable.

A I first mapped where the systems agreed and where they genuinely differed. I preserved source identifiers for traceability, introduced stable warehouse identities, normalized types and lifecycle behavior, and isolated source-specific logic before bringing both paths into common models. I validated each domain with paired examples so downstream teams could see that “unified” did not mean erasing meaningful differences.

R Analysts could work from one coherent model instead of learning two platforms, and the team gained a reusable integration pattern rather than a merger-specific patchwork.

hhax-11

Diagnosed and remediated a multi-payer EVV dashboard data-loss defect by tracing fact-to-dimension joins across 1.12 billion visit records; quantified 214.5 million records (19.2%) excluded by missing patient, service-code, and payer mappings, isolated 99.2% of in-scope missing-patient visits to the CareSource feed, and corrected the underlying integration defects.

S A multi-payer EVV dashboard was visibly undercounting visits, and the first suspicion was a reporting filter. Changing the dashboard would have hidden the symptom without proving whether the warehouse had already discarded the records.

T I needed to find exactly where the population disappeared and determine whether the problem was broad or concentrated in one integration.

A I treated the join path as a funnel, measuring which visits survived each patient, service-code, and payer relationship. That showed roughly one in five visits falling out before the dashboard ever saw them. I then narrowed the missing-patient cohort by payer and source and found that it was overwhelmingly concentrated in the CareSource feed. I traced representative records back to the faulty mappings and corrected the patient, service-code, and payer integration logic.

R We fixed the data loss at its source rather than disguising it in the dashboard, restored confidence in the multi-payer view, and gained a reusable method for investigating future join loss.

hhax-12

Executed structural, row-level, transformation, and completeness QA for a 156-field Snowflake/dbt EVV visit model; uncovered and corrected defects associated with a 52.23% source-to-target row-count gap, 73.6% null patient identifiers, and 105.6 million orphaned visit relationships.

S A new Sandata visit model completed successfully and looked structurally plausible, but downstream analytics would depend on every visit retaining its patient and business context. A green dbt run was not evidence that the population had survived.

T I needed to prove the model’s grain, transformations, and completeness before anyone built reporting on top of it.

A I worked from the outside in: verified the schema and field behavior, tested visit identity, reconciled the source population, and then followed relationships into the patient model. The turning point came when the reconciliation showed that more than half of the expected visits were missing and that null patient identities and orphaned relationships were systemic, not edge cases. I traced the losses through the join and mapping logic, corrected the responsible transformations, and reran the same acceptance sequence.

R The team avoided publishing a model that looked complete but was analytically unsafe, and the repaired visit layer moved forward with a reproducible body of evidence behind it.

hhax-13

Validated and remediated an EVV patient-address model, resolving six documented defects involving deletion handling, multi-address coverage, usage-tag logic, and production data types; verified 1.31 million unique keys, zero missing parent references, and exact 158,084-row parity for the deduplicated parent-matched population.

S The first patient-address model appeared workable for a single address, but real patients could have multiple addresses with different purposes and lifecycle states. The model also failed in production when an upstream type changed, revealing that both its business logic and operational resilience needed attention.

T I needed to redesign the model around the patient relationship, correct the address semantics, and prove that the production fix did not introduce new data loss.

A I documented each failure mode, reused the governed patient entity, incorporated the appropriate supplemental address sources, added deletion and usage semantics, aligned the key pattern with the established HHA model, and delivered a focused type-handling hotfix. I then validated uniqueness, parent linkage, required fields, join coverage, filters, and source-to-target reconciliation as one acceptance path.

R The model supported real multi-address behavior, every retained address resolved to a patient, the reconciled population matched its source exactly, and the production issue was closed without sacrificing analytical correctness.

hhax-05

Conducted feasibility analysis of source-system structures and implementation constraints to shape integration scope, modeling strategy, and delivery approach.

S The team was under pressure to commit to a unified EVV design before anyone had proven that the Sandata snapshots contained the history and relationships the existing analytics required. The risk was promising parity and discovering late that the source could not support it.

T I needed to separate what was genuinely feasible from what required a changed scope or an explicit limitation.

A I treated the source like an investigation rather than a specification. I profiled raw behavior, followed candidate joins, challenged assumed grains, and tested how updates and deletions appeared over time. I brought the gaps back as concrete examples and used them to shape staging boundaries, identity rules, and acceptance criteria before the full build began.

R Stakeholders made scope decisions with evidence, engineering avoided designing around false assumptions, and known source limitations became planned constraints instead of late surprises.

hhax-04

Translated ambiguous business requirements into documented mappings, acceptance criteria, onboarding standards, and 12 data-quality investigation runbooks.

S Requirements often arrived as a simple request to make Sandata data “work like HHA.” That phrase concealed important choices about grain, joins, lifecycle states, and how much variance was acceptable. Different reviewers could approve different behavior while believing they agreed.

T I needed to turn an ambiguous comparison into a shared definition of done.

A I translated each business expectation into a mapping, an acceptance criterion, and an executable investigation path. The resulting runbooks led an engineer from symptom to evidence: what to query, what relationship to expect, how to classify severity, and what had to be retested before sign-off. I used real failure examples so the documents reflected how investigations actually unfolded.

R Reviews became faster and less subjective, disagreements surfaced while they were still design questions, and new team members had a practical playbook for diagnosing future data problems.

hhax-07

Defined surrogate-key, natural-key, and crosswalk strategies supporting stable identity resolution across related source systems.

S Both source systems reused local identifiers. Two unrelated patients or offices could share the same numeric ID simply because they belonged to different accounts, which meant a naive join could create a believable but incorrect relationship.

T I needed identities that were stable across the warehouse without severing the path back to the source record.

A I defined the true business grain for each entity, built composite natural keys with the account and domain context needed to make them meaningful, and mapped those keys to stable warehouse identifiers. I carried the same rules through facts, dimensions, and crosswalks, then deliberately searched for collisions and broken references.

R Shared models stopped confusing locally reused IDs, relationships remained traceable to their origin, and later identity corrections could be made in one governed place.

hhax-06

Structured target models using FHIR-like concepts where appropriate to improve consistency and extensibility for downstream integration and analytics.

S In the merged environment, the same healthcare concept appeared under different names and table structures. Downstream users had to understand each source before they could answer a basic question about a patient, caregiver, location, or address.

T I needed a target model that felt familiar across systems and could evolve without falsely claiming full FHIR conformance.

A I borrowed FHIR-like resource boundaries where they clarified the data, separated core entities from relationships and supporting attributes, and preserved provenance at every boundary. I repeatedly checked the conceptual design against actual report fields and join paths, rejecting abstractions that made the real workflow harder to express.

R The warehouse acquired a more intuitive vocabulary, mappings became easier to review, and future integrations had recognizable extension points instead of another source-shaped schema.

hhax-02

Implemented reusable SQL/Jinja transformation logic for lifecycle states, null handling, sentinel dates, and source-system variations across organization, office, caregiver, and patient domains.

S The most dangerous differences between the HHA and Sandata feeds were not dramatic schema changes; they were small semantic disagreements. A blank value could mean “unknown” in one feed and “not applicable” in another, while an inactive or deleted record could remain apparently current if interpreted literally.

T I needed to make those differences predictable before they reached shared patient, caregiver, office, and organization models.

A I turned recurring edge cases into reusable SQL/Jinja rules for types, nulls, dates, lifecycle states, and deterministic record selection. I kept source labels and application IDs visible, so anyone investigating an exception could follow it back to the originating system rather than treating the shared model as a black box.

R New domains inherited consistent behavior, reviewers could explain why a record was active or excluded, and source variation stopped leaking into every downstream model.

hhax-03

Created or materially redesigned 30 automated SQL/dbt data-quality tests covering nulls, duplicates, referential integrity, business rules, lifecycle logic, and source-to-target reconciliation.

S New EVV models were progressing through environments, but review still depended too heavily on spot checks. A model could compile successfully while quietly duplicating patients, breaking parent-child relationships, or mishandling a deletion state.

T I needed to turn those hidden risks into failures that appeared early and explained themselves.

A I built a layered dbt test suite covering structural expectations, business rules, relationships, lifecycle behavior, and source-to-target reconciliation. When built-in tests could not handle quoted identifiers or source-specific rules, I created reusable generic tests. I also reran the exact failing cohort after each fix so a green result represented regression evidence, not a fresh sample.

R Quality review became a repeatable gate rather than a judgment call, defects reached engineers with a clear failure mode, and releases could proceed with evidence instead of optimism.

hhax-09

Collaborated with client and internal stakeholders to resolve source ambiguity, refine requirements, and translate business needs into production-ready data assets.

S Some of the most important EVV rules could not be discovered from column names. The data alone could not tell us which patient address was authoritative, what a usage tag meant, or which lifecycle event downstream reporting considered final. Guessing would have produced clean-looking but incorrect models.

T I needed to get the right business decision without asking stakeholders to reason from abstract schemas.

A I brought concrete patient and visit examples, showed how each possible interpretation changed the output, and framed the decision in operational terms. Once stakeholders chose the intended behavior, I captured it in mappings, acceptance criteria, and QA evidence, incorporated peer feedback, and demonstrated the result in Snowflake before promotion.

R Ambiguity became an explicit, testable rule, stakeholders understood the consequence of the choice, and production logic had a documented business rationale.

hhax-08

Resolved data-quality and mapping issues through targeted test-case analysis and review of failing source and target records.

S Integration defects rarely announced where they began. A missing dashboard record might originate in a raw source value, a staging cast, a deduplication rule, or a join several layers upstream. Broad fixes risked disturbing data that was already correct.

T I needed to find the first point where a known-good record changed or disappeared and correct only that behavior.

A I built small cohorts around representative failures and followed the same identifiers from raw ingestion through staging, core, and BI. I removed joins one at a time, compared null and population changes, and used the earliest divergence to identify the faulty rule. After making a focused correction, I replayed the original case as a regression test.

R Investigations became explainable rather than speculative, fixes stayed narrowly scoped, and reviewers could see both the defect and the evidence that it was gone.

hhax-10

Established reusable patterns for onboarding additional sources and extending integration workflows.

S The merger was only the immediate integration challenge. If the solution depended on hard-coded exceptions, the next acquired or partner source would force the team to solve the same architectural problem again.

T I needed to leave behind an onboarding path that another engineer could understand and repeat.

A I standardized the movement from staging to core to BI, localized source-specific logic behind a common contract, and made identity fields, provenance, tests, runbooks, and versioned database changes part of the adapter pattern. I wrote the pattern from the perspective of the next engineer: where a new source should vary, what must remain invariant, and what evidence was required before release.

R Future onboarding became an exercise in implementing a known contract rather than redesigning the warehouse, reducing both delivery risk and dependence on the original authors.

GrowData Analytics – Principal ML Engineer

Dates: Sep. 2024 - Oct. 2025
Technologies: Python, Ruby, RubyLLM, RSpec, RuboCop, Tesseract OCR, LLM APIs, Docker, AWS, GitHub Actions

Founded GrowData Analytics to explore how AI led analytics could help small local farms make better operational decisions. The work exposed the more fundamental obstacle that the source data was trapped in inconsistent invoices. I built a preproduction document-to-data product with OCR, validation, audit trails, operational analytics, and a lightweight AWS deployment.

`growdata-01`

Built and validated a preproduction document-to-data MVP that converted semi-structured invoices into auditable transaction records, achieving 95% field-level accuracy across a stratified 500-invoice test suite.

S Small local farms had valuable purchasing and operating data trapped in handwritten and printed invoices whose layouts and image quality varied widely.

T I needed to prove that a document-to-data product could produce useful records without sacrificing accuracy or auditability.

A I assembled a stratified 500-invoice test suite across representative vendors, layouts, and quality levels; combined image preprocessing, OCR, language-model extraction, a versioned JSON schema, and arithmetic reconciliation; and compared outputs field by field while iterating on prompts and post-processing rules.

R The preproduction MVP achieved 95% field-level accuracy and produced structured transaction records that retained enough context for review and correction.

N **Notes / audit prompts**

- **MVP (minimum viable product):** the smallest end-to-end implementation that can test whether the proposed product creates real value.
- **OCR (optical character recognition):** converts pixels in an invoice image into machine-readable text; it is analogous to an ingestion parser for documents.
- **Language-model extraction:** maps noisy OCR text into named business fields such as vendor, date, line item, quantity, and total. The model proposes values; deterministic validation decides whether they are acceptable.
- **Stratified test suite:** a benchmark deliberately sampled across vendors, layouts, handwriting, printing, and image quality so one easy document type cannot dominate the accuracy result.

growdata-02

Instrumented the MVP with automated tests, audit logging, retry controls, and confidence routing, achieving 88% straight-through processing and under 1% system failure.

S High average extraction accuracy was not sufficient for an operational product because uncertain invoices and transient provider failures still needed safe handling.

T I needed to maximize unattended processing while ensuring questionable records were reviewed rather than silently accepted.

A I added automated tests and audit events, bounded retry controls for transient failures, validation and reconciliation signals, and confidence-based routing that separated clean records from invoices requiring human review. I tracked system failures independently from data-quality exceptions so each could be improved appropriately.

R The MVP reached 88% straight-through processing while holding system failure below 1%, with an explicit review path for the remainder.

N **Notes / audit prompts**

- **Straight-through processing (STP):** a document completes ingestion, extraction, validation, and persistence without human intervention.
- **Confidence routing:** combines extraction confidence and deterministic checks to send reliable records forward and questionable ones to a review queue. It is comparable to routing invalid or suspect data into quarantine instead of a trusted table.
- **Bounded retry:** retries a transient failure only a limited number of times, usually with delay or backoff, so a failing dependency cannot create an endless loop.
- **System failure versus data-quality exception:** a timeout or unavailable provider means the pipeline failed to execute; an implausible total or missing vendor means it executed but produced data that should not be accepted automatically.
- **Audit event:** a structured record of what ran, which versions were used, what checks passed, and why the invoice was accepted, retried, or routed for review.

growdata-04

Added schema checks, cross-field validation, reconciliation, idempotent reruns, fixtures, golden-file tests, linting, and release automation.

S Changes to OCR, prompts, or post-processing could improve one invoice while breaking required fields, totals, or previously correct layouts.

T I needed a release gate that protected the transaction contract and made reprocessing safe.

A I enforced a versioned JSON schema, added cross-field and line-to-total reconciliation, designed idempotent reruns, built representative fixtures and golden-file regression tests, and connected tests and linting to release automation.

R Changes could be evaluated against known invoices before release, malformed or inconsistent records were stopped early, and failed work could be rerun without creating duplicate transactions.

N **Notes / audit prompts**

- **Cross-field validation:** checks relationships between fields, such as a due date following an invoice date or a line amount agreeing with quantity times price.
- **Reconciliation:** independently recomputes subtotals, tax, and totals to detect plausible-looking extraction errors.
- **Idempotent rerun:** processing the same invoice again produces or updates the same logical transaction rather than inserting a duplicate.
- **Fixture:** a controlled test input with known characteristics. A **golden file** is the reviewed expected output used for regression comparison.
- **Linting and release automation:** static checks and CI steps that prevent inconsistent code or failed tests from being promoted into a release.

growdata-03

Productized Tesseract OCR and rule-based post-processing as a small Python/Ruby service with containerized deployments and versioned configuration.

S Early OCR experiments worked as isolated scripts but were sensitive to machine setup, image preparation, and unrecorded parameter changes.

T I needed to turn that experimentation into a reproducible component that could be operated and improved like a product.

A I wrapped Tesseract behind an extraction interface, added repeatable image orientation, grayscale, contrast, and normalization steps, applied rule-based cleanup, exposed the workflow through a small Python/Ruby service, and packaged its dependencies and versioned settings in containers.

R The same invoice and configuration produced consistent behavior across environments, making OCR changes easier to test, deploy, and roll back.

N **Notes / audit prompts**

- **Tesseract:** an open-source OCR engine that recognizes characters but does not by itself understand invoice fields or business meaning.
- **Image preprocessing:** deterministic transformations such as rotation, grayscale conversion, contrast adjustment, resizing, and noise reduction that improve the input before recognition.
- **Rule-based post-processing:** ordinary code that normalizes dates, currency, whitespace, vendor spellings, or common OCR substitutions after recognition.
- **Service wrapper:** a stable interface around the OCR workflow so callers do not need to know its command-line arguments, temporary files, or implementation details.
- **Container:** packages the runtime, libraries, OCR binary, and configuration so the same build can run consistently on a laptop, CI worker, or cloud host.
- **Versioned configuration:** preprocessing and OCR settings are reviewed and released like code, making behavioral changes traceable and reversible.

growdata-06

Built a lightweight analytics layer tracking usage, error categories, latency, and throughput to guide feature tuning and cost control.

S Without operational measurements, I could not tell whether a feature improved customer value or merely added latency and model cost.

T I needed enough observability to guide a small product without creating a large data platform.

A I captured usage, latency, throughput, review routing, and categorized error events; summarized them by workflow and configuration version; and compared quality and processing behavior before and after changes.

R The lightweight analytics layer exposed bottlenecks and recurring failure patterns, gave me evidence for feature and configuration tuning, and kept infrastructure and model-spend decisions tied to actual use.

N **Notes / audit prompts**

- **Observability:** structured evidence about what the system is doing, usually through metrics, logs, traces, and categorized events.
- **Latency:** elapsed time to process one request or document.
- **Throughput:** the amount of work completed in a time window, such as invoices per hour.
- **Error taxonomy:** stable categories separating OCR, validation, provider, infrastructure, and user-input failures so trends are actionable.
- **Configuration-version comparison:** the ML equivalent of comparing pipeline releases: quality, cost, and runtime are measured before and after a prompt, model, preprocessing, or rule change.
- **Model spend:** variable cost from external inference calls, tracked per document or workflow so accuracy improvements can be weighed against operating cost.

growdata-09

Led marketing and client acquisition, including creating promotional materials for the business.

S A technically viable invoice parser still needed a clear value proposition and direct feedback from the small farms it was intended to serve.

T I needed to explain the product in practical terms, create interest, and turn outreach into product learning and client opportunities.

A I translated the workflow into benefits such as reduced data entry and better expense visibility, created promotional and demonstration materials, conducted outreach and product conversations, and used objections and requests to refine positioning and the roadmap.

R GrowData established an initial acquisition process and a feedback loop connecting market conversations to product priorities.

N **Notes / audit prompts**

- **Value proposition:** a concise explanation of the customer problem, the benefit delivered, and why the product is preferable to the current process.
- **Client acquisition:** the path from identifying a prospective user through outreach, qualification, demonstration, follow-up, and a decision to pilot or buy.
- **Positioning:** the language and comparison that help a specific audience understand what the product is for; farmers may care more about saved data-entry time and expense visibility than the underlying OCR or model architecture.
- **Feedback loop:** objections, requests, and observed workflow friction are categorized and fed into prioritization rather than treated as isolated conversations.
- **Roadmap:** an ordered set of product outcomes and experiments, not merely a feature list; market evidence helps determine what should be built next.

growdata-07

Managed infrastructure as code for a minimal AWS footprint using S3, Lambda or scheduled jobs, and CloudWatch alerts through Bash/Zsh and Makefile workflows.

S As a self-funded preproduction product, GrowData needed reliable execution and monitoring without the cost or operational burden of a large platform.

T I needed to deploy the pipeline in a minimal, reproducible AWS footprint.

A I managed the infrastructure as code, used S3 for controlled document and result storage, selected Lambda or scheduled execution according to workload needs, configured least-privilege access and CloudWatch alerts, and wrapped common deployment and operating steps in Bash/Zsh and Makefile workflows.

R The service could be recreated and monitored consistently, remained inexpensive at preproduction volume, and avoided manual environment configuration.

N **Notes / audit prompts**

- **IaC (infrastructure as code):** cloud resources and configuration are declared in versioned files or repeatable scripts instead of being created manually.
- **S3 (Simple Storage Service):** AWS object storage used for source images, intermediate artifacts, and results; conceptually similar to a cloud data-lake landing zone.
- **Lambda:** an event-driven AWS compute service suited to short, isolated jobs. Scheduled execution is preferable when work is batch-oriented or exceeds Lambda limits.
- **CloudWatch:** AWS logging, metrics, dashboards, and alerting used to make failures and unusual runtime behavior visible.
- **Least privilege:** each process receives only the permissions it needs, such as access to a specific bucket path rather than broad account access.
- **Makefile and shell workflows:** thin, repeatable operator commands for building, testing, deploying, and inspecting the service.

growdata-05

Documented runbooks and failure modes covering timeouts, low-confidence OCR, and retries.

S The pipeline depended on image quality, local OCR, and external model APIs, so failures ranged from slow requests to readable but low-confidence results.

T I needed predictable recovery procedures that did not depend on remembering how a particular failure had been solved before.

A I cataloged timeout, malformed-response, low-confidence OCR, validation, reconciliation, and exhausted-retry scenarios; documented the relevant logs and checks; and defined when to retry, preprocess again, route to review, reprocess, or escalate.

R Incidents could be classified and handled consistently, limiting blind retries and reducing the chance that an uncertain invoice would enter the transaction dataset.

N **Notes / audit prompts**

- **Runbook:** an operational decision tree describing symptoms, evidence to inspect, recovery steps, escalation conditions, and how to verify resolution.
- **Timeout:** the dependency did not respond within the allowed duration; this is often transient and may justify a bounded retry.
- **Malformed response:** the provider returned content that could not be parsed or did not match the expected schema; retrying unchanged input may not help.
- **Low-confidence OCR:** the text is readable enough to produce candidates but too uncertain for unattended acceptance, so the document should be preprocessed differently or routed to human review.
- **Exhausted retry:** all permitted attempts failed; the work is moved to a visible failure state rather than silently dropped or retried forever.

growdata-08

Managed the business's bookkeeping, accounting, and tax obligations.

S Building the product as a solo principal also made me responsible for the financial records and compliance work behind the business.

T I needed to maintain an accurate view of cash activity and meet filing obligations while continuing product development.

A I separated and categorized business income and expenses, reconciled accounts, retained source documents, maintained bookkeeping records, monitored upcoming obligations, and prepared the information needed for accounting and tax filings.

R The business maintained organized, current records and met its administrative obligations without requiring a separate operations team.

N **Notes / audit prompts**

- **Bookkeeping:** maintaining the detailed transaction ledger for income, expenses, assets, liabilities, and owner activity.
- **Account reconciliation:** comparing the ledger with bank, card, and payment-provider statements and resolving every difference.
- **Source document:** the receipt, invoice, statement, or filing that supports a ledger entry and provides an audit trail.
- **Accounting versus bookkeeping:** bookkeeping maintains accurate records; accounting interprets them for financial statements, decisions, and filings.
- For a data engineer, the analogy is a governed financial pipeline: source evidence, classified transactions, balancing controls, traceability, and period-end outputs.

Adonis – Senior Data Engineer

Dates: Mar. 2025 - Jul. 2025
Technologies: AWS S3, Snowflake, SQLMesh, SQL, Python, Docker, GitHub Actions

Joined a short-turnaround engagement to onboard Hopebridge Autism Therapy Center onto a revenue-cycle data platform. Transformed poorly documented source data into 54 tested Snowflake models, resolved fundamental key usage problems, and delivered a reusable onboarding and validation framework. Translated payer and revenue-cycle source data into standardized Snowflake models for repeatable client onboarding, analytics, and reporting. Contract via Health Data Movers.

adonis-03

Diagnosed undocumented source-key versioning and implemented composite business grains with surrogate keys across 10 charge and payment models, eliminating duplicate-key failures.

S Practice Suite reused or regenerated identifiers as claims were rebilled and payments were posted, but that versioning behavior was not documented; identifiers that appeared unique at first produced duplicate-key failures when the full history was modeled.

T I needed to establish the true business grain for charge and payment data without dropping legitimate activity.

A I profiled the duplicate cohorts, traced claim numbers back to the stable accession number, and tested combinations of claim, line, charge, payment, payer, balance, and transaction attributes. I encoded the resulting composite grains and deterministic SHA-256 surrogate keys across 10 charge, payment, remittance, and transaction models, then propagated the keys through downstream relationships and reran uniqueness and reconciliation checks.

R The changes eliminated the duplicate-key failures while preserving distinct rebills and payment events, giving downstream joins stable claim-, service-line-, and transaction-level identities.

adonis-01

Built, documented, validated, and maintained 54 SQLMesh models that delivered standardized revenue-cycle datasets for a provider whose network served 4,598 families across 113 locations and processed 86,388 annual authorizations.

S Hopebridge needed a new Practice Suite integration on a short timeline, but the source arrived as a poorly documented maze of database tables whose fields did not map cleanly to the Adonis revenue-cycle model.

T As the engineer doing the main ETL and coding work, I needed to turn that raw source into a complete, supportable data product for claims, worklists, reporting, and downstream analytics.

A I built, documented, tested, and maintained 54 SQLMesh models—32 intermediate transformations, 14 core entities, and eight intelligence aggregates—covering patients, providers, organizations, claims, service lines, transactions, remittances, and balances. I reconciled the outputs to source records and to workflows reviewed with the client and product team.

R I delivered standardized Snowflake datasets for a network serving 4,598 families across 113 locations and processing 86,388 annual authorizations, with a structure another engineer could operate and extend.

adonis-02

Implemented automated row-count, uniqueness, and not-null audits across 54 models, achieving a 100% pass rate across 117+ configured audit applications at final delivery.

S A successful SQLMesh plan only proved that the transformations executed; it did not prove that they preserved the intended population, respected each model's grain, or populated required fields.

T I needed an objective release gate that would expose data defects before Hopebridge used the integration in production workflows.

A I attached reusable uniqueness, not-null, composite-key, and source-to-target row-count audits to the Practice Suite models, including a threshold-based comparison audit for complex transformations. I also created a model-by-model QA runbook covering field population, referential integrity, allowed values, dates, and representative business rules, then investigated failures and reran the same checks after each correction.

R At final delivery, all 117+ configured audit applications passed, providing reproducible evidence that the 54-model integration met its structural and completeness expectations.

adonis-05

Built a reusable SQLMesh/Snowflake ingestion and transformation framework using templated models, data contracts, CI checks, and promotion gates.

S The broader integration repository was moving from legacy, client-specific implementations toward a shared SQLMesh structure, while Practice Suite still required source-specific mappings and client schemas.

T I needed to implement Hopebridge in a way that fit the common framework and could be reused instead of becoming another isolated pipeline.

A I organized the work into intermediate, core-entity, and intelligence layers; parameterized database and client schemas; reused macros for identifiers and aggregates; declared grains, column contracts, and audits with the models; and exercised SQLMesh plans in isolated environments before promotion. I also reviewed patterns used by other integrations and fed implementation lessons back into the shared approach.

R Practice Suite became a repeatable SQLMesh/Snowflake project with source-specific logic localized behind a consistent contract, allowing safer CI checks and promotion while giving later client work a practical reference design.

adonis-08

Standardized transformation logic across clients and translated business requirements into technical specifications to improve delivery predictability.

S Important Practice Suite behavior was encoded in client workflows rather than the database documentation: collection status carried different meaning from claim status, rebills received new claim numbers, accession numbers remained stable, and requested fields such as taxonomy were not always in the delivered source.

T I needed to convert those discoveries into precise implementation rules while keeping them compatible with shared Adonis data contracts.

A In client and product working sessions, I used concrete records and screens to ask how rebills, submission dates, worklists, and identifiers were actually used. I translated the answers into model grains, mappings, filters, required-field gaps, audit criteria, and technical specifications, and compared the approach with peer integrations during review.

R Business ambiguity became testable transformation logic, the team could identify missing vendor data early, and shared patterns improved delivery predictability without erasing the source-specific rules Hopebridge depended on.

adonis-04

Authored implementation playbooks, technical handoff documentation, and a QA runbook for a repeatable healthcare-data validation workflow.

S The engagement combined a compressed delivery schedule with a source system that required extensive discovery, so undocumented knowledge about setup, mappings, validation, and recovery would have made the integration difficult to support after handoff.

T I needed to make both the implementation process and its quality checks repeatable for teammates working on Practice Suite and other integrations.

A I documented the SQLMesh project setup, layered model conventions, key and audit patterns, plan and run workflow, and troubleshooting steps. I created a detailed QA runbook for 14 core subject areas with required audits, field-population checks, referential-integrity tests, business rules, and items to preserve for future maintainers, and I used the same structure during peer review and handoff.

R The team received an executable validation and support playbook rather than tribal knowledge, improving the consistency of reviews, onboarding, and post-delivery troubleshooting.

adonis-06

Added schema-drift alerts, parity checks, reconciliation logs, deterministic rerun procedures, and incident-handling guidance for UNIX-like environments.

S Practice Suite data and SQLMesh plans could change independently, and a rerun after an upstream schema change or partial failure risked producing inconsistent results without a clear recovery path.

T I needed to make the integration observable and safe to rerun in a UNIX-like operating environment.

A I added schema and contract checks, source-to-target parity tests, and reconciliation logging around model execution; used deterministic composite and surrogate keys so the same inputs recreated the same identities; and documented how to inspect SQLMesh logs, isolate the failed model or audit, correct the cause, rerun in a development environment, and promote only after the checks passed. I coupled those controls with alerts for material drift and failed validation.

R Schema changes and incidents became diagnosable events with a repeatable recovery sequence, reducing the risk that a partial rerun or silent drift would reach downstream reporting.

adonis-07

Packaged a lightweight operations forecasting and alerting pattern as a tested Python and Docker service with GitHub Actions deployment automation.

S The team needed earlier warning when operational volume or processing behavior moved outside expected ranges, but a heavyweight platform change was not appropriate for the engagement.

T I needed to create a small, portable forecasting and alerting capability that could be tested and deployed consistently.

A I implemented the pattern as a Python service that derived an expected operating range from historical observations, compared new results with that baseline, and emitted actionable alerts for material deviations or failed runs. I added automated tests for normal, threshold, and failure paths, packaged the service in Docker, and configured GitHub Actions to validate and deploy the same artifact.

R The team gained a reusable, low-overhead monitoring component that converted operational variance into an early signal and could be promoted without environment-specific manual steps.

Arkos Health – Senior Data Analyst

Dates: Jun. 2023 - Jan. 2025
Technologies: AWS S3, Athena, Redshift, Glue, SNS, dbt, PySpark, Docker, Jupyter, Power BI, Tableau

Joined as a senior data analyst supporting Medicare risk capture, then grew into the data engineer and platform builder the team needed. Built a greenfield AWS data-lake proof of concept using Glue, Athena, event notifications, and SNS. After validating the approach, designed and built the final architecture spanning the data lake and Redshift, using dbt as the transformation and modeling layer.

arkos-01

Architected and built a greenfield AWS data-lake foundation integrating 18 source systems through approximately 200 scheduled data assets and 200 dbt models, supporting six analysts across six state markets; CI/CD automation and targeted model refactoring accelerated deployments and improved query performance.

S Arkos needed to replace fragmented, source-specific reporting with a governed platform that could support six analysts and six state markets.

T I was responsible for proving the architecture and turning it into a production data lake that could absorb 18 operational, clinical, and payer sources.

A I designed the S3 raw, bronze, staging, silver, and gold layers; built Glue, Athena, Redshift, and dbt processing; standardized configuration and scheduling; and introduced CI/CD and targeted refactoring as the model estate grew.

R The platform scaled to roughly 200 scheduled assets and 200 dbt models, gave analysts repeatable governed datasets, and made releases and performance improvements safer to deliver.

arkos-19

Configured event notifications for the greenfield data-lake CI/CD workflows, routing workflow status and failure events through Amazon SNS for centralized tracking.

S As automated deployments expanded, workflow outcomes were visible in individual AWS or CI/CD consoles but were not collected in one place.

T I needed to make deployment and failure states observable without requiring analysts to continually inspect each workflow.

A I wired workflow status and failure events into Amazon SNS, standardized the event details needed for triage, and validated that successful and failed executions produced usable notifications and audit records.

R The team gained a central signal for CI/CD activity, could identify failed runs more quickly, and had the context needed to begin follow-up without first reconstructing the execution path.

arkos-20

Established SNS topics and subscriptions for CI/CD workflow notifications, giving the analytics team timely visibility into deployment outcomes and supporting follow-up on failed runs.

S Producing workflow events alone did not ensure that the right people would receive actionable information at the right time.

T I needed to turn those events into a dependable notification channel for the analytics team.

A I created and organized SNS topics and subscriptions for CI/CD outcomes, connected publishers and consumers, tested delivery and failure behavior, and documented how the team should use the notifications during support.

R Analysts received timely deployment and failed-run visibility, reducing the chance that a broken workflow would go unnoticed and establishing a repeatable path from alert to investigation.

arkos-18

Built and maintained the risk-adjustment SQL/dbt model inventory; supported three RAF notebooks, their companion workbook, and the Tableau Risk Dashboard while preserving an auditable record of workflow logic, inputs, outputs, and provenance.

S Risk-adjustment logic was distributed across SQL/dbt models, three team-authored RAF notebooks, a companion workbook, and a Tableau dashboard, which made dependencies and ownership easy to blur.

T I needed to make the backend model estate supportable while preserving accurate provenance.

A I built and maintained the SQL/dbt inventory, documented inputs, outputs, dependencies, and business rules, and helped notebook, workbook, and dashboard owners reconcile their results to the governed models.

R The team had an auditable chain from source data to analytical output, could diagnose mismatches more quickly, and retained a clear distinction between my model work and the presentation assets built by others.

arkos-16

Built the SQL/dbt ingestion and normalization layer used by version-aware AWS Glue and Athena pipelines for evolving MAO, HCC, MOR, and related payer files, preserving source lineage and standardizing downstream datasets in partitioned Parquet across the lakehouse.

S MAO, HCC, MOR, and payer files arrived in changing layouts and versions, which could break downstream risk models or silently shift field meaning.

T I needed to absorb those variations while presenting stable analytical datasets.

A I built source- and version-aware Glue and Athena ingestion, normalized the outputs through SQL/dbt contracts, preserved file and row lineage, handled latest-record logic, and wrote partitioned Parquet into governed lakehouse layers.

R Downstream models consumed consistent structures despite upstream format changes, and analysts could trace normalized records back to the originating file and version.

arkos-10

Built SQL/dbt models for multi-market Medicare RAF data preparation in Arizona, North Dakota, and Illinois, combining enrollment and MMR demographic factors with medical claims, signed EHR diagnoses, and payer encounter and MAO diagnoses; generated member and diagnosis inputs supporting CMS-HCC scoring and pre-/post-wellness-visit RAF analysis.

S Arizona, North Dakota, and Illinois had different payer feeds and operational patterns, while RAF analysis required a consistent view of member eligibility, demographics, and diagnoses.

T I needed to prepare reliable member and diagnosis inputs for CMS-HCC scoring and pre-/post-wellness analysis.

A I built SQL/dbt models that joined enrollment and MMR factors to claims, signed EHR diagnoses, payer encounter data, and MAO results; aligned member, contract, and service-date keys; and reconciled market-specific exceptions.

R The risk team received comparable multi-market inputs and could evaluate RAF movement using a documented data chain instead of separate ad hoc extracts.

arkos-11

Built SQL/dbt models implementing diagnosis-submission lineage and acceptance logic covering add/delete activity, repeated submissions, encounter disposition, source and date-of-service preservation, and selection of the latest accepted diagnosis state.

S A diagnosis could be submitted repeatedly, added or deleted, and receive different encounter dispositions over time, making a simple latest-row query unreliable.

T I needed to determine the defensible current state without losing the history required for audit.

A I defined the diagnosis grain and ordering rules, preserved source and date of service, modeled add/delete and repeated-submission activity, incorporated encounter disposition, and selected the latest accepted state while retaining lineage.

R Analysts could explain why a diagnosis was counted or excluded and trace the answer back through each submission event.

arkos-14

Built SQL/dbt reconciliation models that traced diagnoses from signed EHR encounters through Waystar claims, supplemental files, and MAO-004-style payer and CMS responses, enabling the risk team to isolate where diagnoses dropped from the submission path.

S When a diagnosis appeared in a signed EHR encounter but not in a payer or CMS result, the risk team could not readily see where it left the submission chain.

T I needed to make each handoff reconcilable.

A I built SQL/dbt models that linked signed EHR diagnoses to Waystar claims, supplemental outputs, payer responses, and MAO-004-style CMS responses; standardized matching keys and assigned stage-specific statuses for missing and accepted records.

R The risk team could isolate the point of failure, investigate the appropriate owner or file, and explain the final disposition with end-to-end lineage.

arkos-12

Built SQL/dbt models for diagnosis-level HHS-HCC capture analytics that mapped ICD-10 diagnoses from signed face-to-face visits, reconciled them with payer gap reports, classified existing, new, and net-new capture, identified the first capture source, and produced monthly capture and HCC-per-visit measures.

S Marketplace risk teams needed to know whether diagnoses from signed face-to-face visits actually closed payer-identified HHS-HCC gaps and which encounters created new value.

T I needed to turn visit and gap data into diagnosis-level capture measures.

A I mapped ICD-10 diagnoses to HCCs, reconciled signed visits with payer gap reports, classified existing, new, and net-new capture, identified the first capture source, and aggregated monthly capture and HCC-per-visit measures.

R Stakeholders could distinguish activity from true incremental capture and evaluate performance with traceable diagnosis-level evidence.

arkos-13

Built SQL/dbt models and datasets for supplemental diagnosis, claims, and member-enrollment workflows serving Allwell, NextBlue, Humana, Allymar, and Reveleer, joining claims to the appropriate member, MBI, enrollment period, contract, demographic segment, and member-month and formatting outputs to downstream specifications.

S Allwell, NextBlue, Humana, Allymar, and Reveleer each required different supplemental diagnosis, claims, or enrollment outputs, but all depended on accurate member and coverage alignment.

T I needed to make those deliveries repeatable without duplicating core business logic for every recipient.

A I built shared SQL/dbt preparation models, joined claims to the correct member, MBI, enrollment period, contract, demographic segment, and member month, then applied recipient-specific formatting and validation rules.

R The team could produce multiple downstream files from a consistent foundation, with clearer QA and less manual reassembly for each delivery.

arkos-15

Built a dbt-based Reveleer chart-chase data product across three Medicare contracts, covering eligible-member and encounter selection, market-specific rules, provider and chart-location resolution, stable chase identifiers, and generation of separate chase and diagnosis files.

S Reveleer chart-chase submissions spanned three Medicare contracts with different eligibility, encounter, provider, and chart-location rules.

T I needed to produce compliant files while keeping every selected member and diagnosis traceable across markets.

A I built a dbt model chain for eligible-member and encounter selection, encoded market-specific exclusions, resolved providers and chart locations, created stable chase identifiers, and generated separate chase and diagnosis outputs with validation checks.

R Arkos had a repeatable multi-contract chart-chase product whose records could be reconciled from source data through the delivered files.

arkos-17

Built multi-market SQL/dbt datasets combining member, diagnosis, MOR, payer-gap, and ICD-to-HCC mapping data to support analysis of the CMS-HCC V24-to-V28 transition.

S The CMS-HCC transition from V24 to V28 changed diagnosis mappings and risk categories, so leadership needed to understand the effect across markets.

T I needed to create a comparable analytical foundation for both model versions.

A I built SQL/dbt datasets joining member, diagnosis, MOR, payer-gap, and ICD-to-HCC mapping data, calculated version-specific classifications, and reconciled records that moved, disappeared, or emerged under V28.

R Analysts could evaluate the transition with consistent multi-market inputs, explain category-level changes, and focus follow-up on the members and diagnoses driving the differences.

arkos-02

Integrated existing Jupyter-based analytics and authored platform, architecture, and operating documentation, enabling six analysts to independently use the governed data lake for recurring, executive, and BI reporting.

S Analysts already had valuable Jupyter-based work, but it depended on individual knowledge and was disconnected from the new governed lakehouse.

T I needed to preserve that analytical value while making the platform understandable and usable without constant engineering assistance.

A I integrated reusable notebook workflows with governed data, documented the platform architecture, environments, model layers, operating procedures, and lineage, and supplied examples for recurring, executive, and BI reporting.

R Six analysts could independently find, query, and extend trusted datasets, which reduced one-off handoffs and made the new platform part of their normal reporting workflow.

arkos-03

Instrumented scheduled pipelines with monitoring, alerting, and more than 10 failure classifications; authored three AWS/dbt incident runbooks supporting rapid detection and recovery.

S A growing schedule of AWS and dbt jobs could fail for very different reasons, and raw error messages did not provide a consistent response.

T I needed to shorten the time from a failed run to a correct recovery action.

A I instrumented execution history, logs, data-freshness checks, and alerts; grouped recurring failures into more than 10 operational classifications; and wrote three incident runbooks covering detection, diagnosis, remediation, and escalation.

R The team could distinguish source, infrastructure, transformation, and data-quality failures more quickly and recover scheduled pipelines through a repeatable process instead of ad hoc troubleshooting.

arkos-06

Expanded cross-departmental collaboration to bring operational context into analytics decisions.

S Technically correct models could still miss how risk, quality, clinical, operations, finance, and BI teams used the data in practice.

T I needed to bring that operational context into design decisions before logic became embedded in production.

A I held working sessions with stakeholders, brought sample records and reconciliation results into the discussion, translated business rules into explicit model logic, and documented decisions and exceptions for reviewers.

R Models reflected real workflows, stakeholders could validate the logic in familiar terms, and downstream teams had greater confidence in the resulting measures and datasets.

arkos-09

Initiated recurring stakeholder-feedback sessions to refine deliverables against business needs.

S Requirements for risk and operational reporting continued to evolve after initial delivery, creating a risk of late rework.

T I needed a reliable way to validate assumptions while the work was still inexpensive to change.

A I initiated recurring feedback sessions, demonstrated prototypes and reconciliations, walked stakeholders through edge cases, captured decisions and open questions, and fed the results back into model and delivery priorities.

R Misunderstandings surfaced earlier, deliverables stayed closer to current business needs, and stakeholders had a regular forum for shaping the work rather than waiting for a final handoff.

arkos-04

Mentored engineers on code review, testing, and pipeline design and authored internal delivery standards.

S As more contributors added pipelines and models, inconsistent grains, tests, naming, and review practices increased maintenance risk.

T I needed to raise delivery consistency while helping engineers make sound design decisions themselves.

A I coached teammates through code reviews, explained the reasoning behind test and modeling feedback, paired on pipeline design, and documented standards for naming, model contracts, validation, documentation, and pull-request readiness.

R Reviews became more predictable, defects were caught earlier, and engineers had a shared set of expectations they could apply before requesting review.

arkos-08

Led goal-setting sessions to establish shared objectives and a consistent tracking cadence.

S The team was balancing platform construction, production support, and stakeholder deliverables, but priorities were not always expressed as shared, measurable outcomes.

T I needed to help the group turn competing work into an aligned plan.

A I facilitated goal-setting sessions, grouped work by business and platform outcome, clarified owners, dependencies, and completion criteria, and introduced a recurring cadence for checking progress and revising the plan.

R Team members had a common view of priorities and tradeoffs, blockers surfaced earlier, and status conversations focused on outcomes rather than disconnected task lists.

arkos-05

Implemented Jira and GitHub workflows to centralize task tracking and improve coordination across the analytics team.

S Requests, implementation details, and review status were spread across conversations and tools, making priorities and ownership difficult to track.

T I needed to create a visible path from a business request to reviewed production code.

A I organized work into Jira epics and issues, connected tickets to GitHub branches and pull requests, introduced consistent issue and review expectations, and aligned workflow states with the team's delivery process.

R The analytics team gained clearer ownership and status, reviewers could trace changes back to their requirements, and handoffs involved less searching and follow-up.

arkos-07

Mentored junior team members in code review, testing, pipeline design, and delivery practices.

S Junior team members were taking on increasingly complex SQL, dbt, and AWS work and needed support beyond one-time corrections.

T I wanted to help them develop an independent method for designing, testing, and shipping reliable changes.

A I paired on model and pipeline design, used reviews to explain grain, edge cases, and test strategy, demonstrated structured debugging, and gradually shifted implementation and review ownership back to them.

R They became more confident diagnosing failures and preparing changes, required less step-by-step support, and contributed work that aligned more consistently with team delivery standards.

Merative – Senior Data Scientist

Dates: Jul. 2022 - Mar. 2023
Technologies: Microsoft Azure, Azure Data Factory, Blob Storage, Azure Functions, SQL Database, Synapse Analytics, Python, SQL, Azure DevOps

Served as the quality and acceptance layer for a large scale healthcare platform migration from HDFS to Azure. Built automated reconciliation, regression, deidentification, and governance controls that brought the tested data to sign-off with no unresolved discrepancies.

`merative-02`

Led HIPAA-compliant deidentification validation across demographic, encounter, and coded clinical data representing approximately 300 million patient lives, achieving a 100% pass rate before sign-off.

S The migration included demographic, encounter, and coded clinical data representing approximately 300 million patient lives, so a deidentification defect could create both privacy exposure and analytical breakage.

T I needed to validate that protected fields were transformed correctly while required relationships and clinical utility remained intact.

A I translated deidentification requirements into field-level tests, checked representative known values and edge cases, scanned for residual identifying patterns, verified consistency across related records, and traced every failure back to its transformation or configuration.

R All in-scope deidentification checks passed before sign-off, providing evidence that the migrated data met the defined HIPAA-compliant handling requirements.

N **Notes / audit prompts**

Plausible field-level tests included:

- direct identifiers such as names, MRNs, account numbers, phone numbers, email addresses, and Social Security numbers were removed, masked, or tokenized;
- dates were shifted or generalized consistently for the same patient while age and encounter intervals remained analytically usable;
- ZIP codes and other geographic fields followed the approved suppression or generalization rules;
- pseudonymous patient and encounter keys remained deterministic across related tables without exposing source identifiers;
- diagnosis, procedure, medication, and laboratory codes retained valid formats and source-to-target meaning;
- free-text and residual-value scans found no recognizable identifiers; and
- null, edge-case, and referential-integrity behavior matched the specification.

merative-01

Engineered a seven-layer Python and SQL migration-acceptance suite with zero unresolved discrepancies at sign-off across tested record counts, partitions, schemas, nulls, duplicates, field values, aggregates, and business rules.

S Merative was moving a large healthcare data platform from HDFS to Azure, and successful job completion alone could not prove that the new platform preserved the source data's meaning.

T I needed to create a repeatable acceptance framework that could support a defensible migration sign-off.

A I built a seven-layer Python and SQL suite that compared source and target record counts, partitions, schemas, null and duplicate behavior, field-level values, aggregates, and business rules. I recorded expected tolerances, produced rerunnable discrepancy reports, and worked each unexplained variance through defect, retest, and closure.

R The tested migration scope reached sign-off with zero unresolved discrepancies and a reproducible evidence trail for the decision.

N **Notes / audit prompts**

The seven layers were most likely:

- record-count parity by table, file, and load cohort;
- partition completeness and boundary coverage;
- schema parity for names, types, lengths, and required columns;
- nullability and missing-value behavior;
- primary-key, uniqueness, and duplicate checks;
- field-level and aggregate-value reconciliation; and
- business-rule and end-to-end workflow validation.

merative-07

Enforced data-governance and security protocols supporting audit-risk reduction and HITRUST readiness.

S Moving regulated healthcare data to Azure changed where data was stored, who could access it, and which controls had to be evidenced for internal review and HITRUST readiness.

T I needed to incorporate governance and security checks into migration acceptance rather than treat them as a separate final review.

A I verified that sensitive datasets followed approved storage and handling patterns, checked role-based access and least-privilege expectations, confirmed encryption and logging controls, reviewed retention and movement rules, and maintained evidence and exception records through remediation.

R The migration closed control gaps before sign-off, reduced audit risk, and produced a more complete evidence set for the organization's HITRUST-readiness work.

N **Notes / audit prompts**

Plausible protocols included:

- role-based access with least-privilege group assignments and periodic review;
- encryption in transit and at rest with managed key and secret handling;
- approved storage zones, data classification, and restricted PHI movement;
- centralized logging, access monitoring, and retained audit evidence;
- documented change control, peer review, and separation of duties; and
- retention, exception, incident-response, and remediation tracking aligned to organizational security policy and HITRUST-readiness evidence needs.

merative-05

Collaborated with multidisciplinary engineering teams to refine architectural designs and preserve workflow continuity and data fidelity.

S The migration crossed teams responsible for the HDFS source, Azure services, data engineering, security, and downstream healthcare analytics, and a local design improvement could disrupt another team's workflow.

T I needed to use validation evidence to help refine the architecture without losing continuity or fidelity.

A I joined design and defect reviews, mapped dependencies and source-to-target contracts, presented concrete reconciliation results and failing examples, clarified acceptable differences, and retested proposed changes across upstream and downstream boundaries.

R Architectural issues were resolved before cutover, teams made decisions from shared evidence, and existing analytical workflows retained the data relationships and behavior they depended on.

merative-04

Validated end-to-end Azure Data Factory pipelines, Blob Storage configurations, and downstream Synapse integrations during the transition from HDFS to Azure.

S Individual transformations could pass while an end-to-end workflow still lost a partition, used the wrong storage path, or delivered incomplete data to Synapse.

T I needed to validate the complete path from HDFS extraction through the Azure analytical target.

A I ran controlled cohorts through Azure Data Factory, verified triggers, dependencies, parameters, retries, and failure behavior, reconciled Blob Storage paths and file or partition counts, and queried the resulting Synapse tables for completeness and transformation accuracy. I repeated the checks after configuration and code changes.

R The team had evidence that each handoff worked as a connected system, not merely as independently successful components.

merative-03

Designed and implemented automated ETL testing and regression suites that detected schema drift and transformation errors early in the migration lifecycle.

S Source schemas and migration transformations continued to change while the Azure platform was being built, so defects discovered only during final acceptance would be expensive to isolate.

T I needed to move quality checks earlier in the delivery cycle.

A I established versioned schema and data-profile baselines, automated checks for columns, types, keys, nulls, duplicates, distributions, and derived fields, and connected the regression suite to repeatable pipeline test runs. Material differences generated focused evidence for the responsible engineering team rather than a generic failed comparison.

R Schema drift and transformation errors were detected closer to the change that caused them, reducing late-cycle investigation and protecting the final acceptance schedule.

N **Notes / audit prompts**
Likely Azure-era tooling included Python with pytest and pandas or PySpark for reusable assertions, SQL reconciliation queries in Synapse, Azure Data Factory debug and test runs, Azure DevOps pipelines for scheduled regression execution, Git for versioned baselines, and Blob or ADLS test fixtures. ADF metadata and schema-drift behavior could be compared with approved contracts, with failures published as pipeline artifacts or work items for engineering review.

merative-06

Documented test plans, platform-migration procedures, and onboarding guidance for future quality-assurance engineers.

S The acceptance process accumulated detailed knowledge about source behavior, Azure configuration, expected differences, and recovery steps that could easily remain with the original migration team.

T I needed to make that process repeatable for future quality-assurance engineers.

A I documented test scope and prerequisites, source-to-target mappings, execution steps, expected results and tolerances, discrepancy triage, rerun and escalation procedures, and the evidence required for sign-off. I organized the material into onboarding guidance that followed the actual validation workflow.

R A new engineer could understand the platform and execute the core checks from documented procedures rather than reconstructing the migration logic from code and conversations.

IBM – Senior Data Scientist

Dates: Nov. 2021 - Jul. 2022

Led healthcare data integration, migration validation, and analytics delivery across clinical, claims, operational, and financial sources, including an HDFS-to-Azure transition spanning more than 150 billion deidentified records.

`ibm-senior-data-scientist-03`

Served as technical lead across electronic health records, insurance claims, scheduling and financial data, social determinants of health, and other relevant sources.

S Provider analytics questions increasingly crossed EHR, claims, scheduling, finance, and social-determinants data, none of which shared a simple common model.

T I needed to lead the integration while preserving each source's meaning and limitations.

A I defined contracts and analytical grains, established identity and time-window rules, documented lineage and caveats, coordinated implementation across owners, and required cross-source reconciliation.

R Analysts received an integrated view they could use without hiding material differences in coverage, timing, or provenance.

`ibm-senior-data-scientist-04`

Developed migration-validation analysis spanning more than 150 billion deidentified health records, reconciling source and target schemas, partitions, values, aggregates, and business rules during the transition from legacy HDFS infrastructure to Azure.

S Moving more than 150 billion deidentified health records from HDFS to Azure risked changing content or business behavior despite successful loads.

T I needed migration evidence at a scale where manual sampling was inadequate.

A I automated source-to-target comparisons for schemas, partitions, counts, field values, aggregates, and business rules; isolated differences by dataset and stage; and tracked unexplained variances through correction and rerun.

R The team could distinguish expected platform differences from defects and make acceptance decisions using repeatable, documented evidence.

`ibm-senior-data-scientist-06`

Supported data and integration work for Pennsylvania's statewide P3N health-information exchange, encompassing master patient identity, provider directories, clinical-document discovery, patient consent, and real-time ADT encounter notifications.

S Pennsylvania's P3N exchange had to connect patient identity, provider directories, consent, document discovery, and real-time ADT notifications across organizations.

T I needed to support the connected workflow rather than treat each function as an isolated feed.

A I mapped identifiers and message flows, validated representative patient and provider matches, checked consent and document-discovery behavior, traced ADT events end to end, and documented exceptions.

R The exchange gained clearer evidence that workflows preserved identity, authorization, and event continuity across boundaries.

N **Notes / audit prompts**

P3N is the Pennsylvania Patient & Provider Network, the Commonwealth's statewide health-information exchange. It connects certified regional HIOs through a shared hub so participating providers can locate and exchange records across organizational boundaries. The network supports functions such as patient and provider matching, consent or opt-out enforcement, clinical-document discovery, and encounter-event notifications; it routes information between connected organizations rather than serving as a central longitudinal record repository.

ibm-senior-data-scientist-13

Engineered a Spark, Hive, and Impala analytics framework powering 50+ prior-authorization lag-time and approval-bottleneck reports with daily operational and periodic regulatory and financial snapshots.

S Prior-authorization status events arrived from multiple systems, sometimes late or out of order, while operations needed current bottleneck visibility and regulatory and financial users needed reproducible historical snapshots.

T I needed one architecture that could calculate request-stage timing continuously and still support governed batch reporting.

A I published authorization events to Kafka, used Flink event-time processing, watermarks, keyed state, and status-transition rules to reconstruct each request and calculate stage and end-to-end lag, and persisted curated results to Hive. Spark jobs reconciled backfills and produced daily and period-end snapshots, while Impala exposed shared dimensions and measures to more than 50 reports.

R Operations received timely, consistent bottleneck signals, and regulatory and financial reports used the same traceable logic and reproducible history.

ibm-senior-data-scientist-02

Worked with senior leaders and colleagues to identify opportunities, define objectives, and formulate analytics strategies that produced measurable impact.

S Senior leaders saw a growing care-gap backlog for a client even though encounter volume and the size of the attributed population appeared stable. They initially proposed a predictive model to prioritize outreach.

T I needed to determine whether modeling was the right intervention and define a measurable analytics strategy.

A I brought clinical, product, and data colleagues together, profiled the attribution and care-gap pipelines, and found that delayed provider-roster refreshes were retaining inactive relationships and inflating the apparent backlog. I reframed the work around roster reconciliation, freshness metrics, and exception reporting before model development.

R The team addressed the data-quality cause first, restored trust in the backlog measure, and avoided investing in a model that would have optimized against a distorted population.

N **Notes / audit prompts**

Assumed example for audit: confirm whether the triggering incident involved a care-gap backlog, delayed provider-roster refreshes, and inactive attribution relationships. If the actual leadership engagement concerned another domain, retain the strategy pattern but replace the client problem, root cause, and resulting decision.

ibm-senior-data-scientist-01

Led Provider Analytics data engineering, profiling disparate EHR and centralized sources, resolving identity and grain conflicts, and building analytics-ready data marts for exploratory and statistical analysis.

S Provider Analytics depended on clinical, claims, and operational sources that used different identifiers, grains, and refresh schedules.

T As a lead member, I needed to create dependable data marts for analysts and modelers.

A I clarified analytical grains, mapped source identities to conformed entities, standardized measures and time windows, built reconciliation checks, and worked with analysts to validate that the marts supported exploratory analysis and statistical-model use cases.

R The team gained reusable, traceable datasets instead of rebuilding source logic for each analysis, shortening the path from question to result.

ibm-senior-data-scientist-07

Supported the controlled decommissioning of legacy Explorys capabilities by analyzing downstream dependencies and validating continuity across normalized clinical, claims, laboratory, billing, and operational datasets.

S Retiring legacy Explorys capabilities risked breaking hidden dependencies across clinical, claims, laboratory, billing, and operational workflows.

T I needed to identify what could be retired safely and what required a replacement path.

A I inventoried consumers, traced lineage to legacy datasets, grouped dependencies by criticality, compared replacement outputs to current behavior, and supported staged validation before shutdown.

R The program could decommission capabilities in a controlled sequence and expose unresolved dependencies before access disappeared.

ibm-senior-data-scientist-08

Supported Cleveland Clinic Foundation's accelerated SuperMart migration from MySQL to Greenplum by assessing dashboard dependencies and defining the changes required to preserve reporting continuity.

S A senior-leadership pivot compressed Cleveland Clinic Foundation's SuperMart migration from MySQL to Greenplum into a five-week window. Preserving the long-standing client relationship also required IBM to modernize the Tableau dashboards that depended on the legacy mart.

T I owned my SuperMart migration work and was solely responsible for converting the Tableau layer, despite not having prior Tableau experience.

A I learned Tableau while inventorying workbook, extract, calculation, filter, and SQL dependencies; translated MySQL-specific logic to Greenplum; rebuilt data connections and dashboards; prioritized the most critical reports; and reconciled visual totals and drill paths against legacy results in tight daily cycles.

R I completed both my migration responsibilities and the Tableau modernization within five weeks, preserving reporting continuity and helping IBM meet the accelerated commitment to Cleveland Clinic.

ibm-senior-data-scientist-12

Led delivery of a tailored analytics and reporting project using client administrative data.

S Cleveland Clinic Foundation had administrative scheduling and encounter data but lacked a unified view of appointment demand, cancellations, no-shows, provider capacity, and time to next available appointment across departments.

T I needed to turn the operational question into a tailored, traceable reporting product rather than a generic utilization dashboard.

A I profiled appointment and encounter status histories, resolved department and provider mappings, defined measures and reporting grains with stakeholders, built a validated analytical mart, and iterated on views that separated patient-driven cancellations from operational rescheduling.

R Leaders gained a consistent way to identify access bottlenecks and distinguish unused capacity from scheduling-process problems, with every metric reconcilable to client administrative records.

N **Notes / audit prompts**

Assumed example for audit: Cleveland Clinic Foundation, ambulatory scheduling and encounter data, and the access-capacity measures are a plausible reconstruction rather than a verified project record. Confirm the client, administrative subject area, audience, and decisions supported.

ibm-senior-data-scientist-05

Provided production and configuration support for Phytel population-health applications, troubleshooting data and operational issues affecting provider configuration, care-gap identification, patient outreach, and quality-reporting workflows.

S Phytel issues could appear as missing care gaps, incorrect provider populations, failed outreach, or reporting discrepancies even when the root cause was configuration or data.

T I needed to restore workflows and identify the right lasting fix.

A I traced examples from provider configuration through patient and measure data, compared expected and actual states, reviewed recent changes, and coordinated corrections across support and engineering.

R Operational issues were resolved with a documented cause, and recurring defects could be prevented through targeted configuration or data changes.

N **Notes / audit prompts**

Typical issues may have included inactive or duplicate provider records, patients attributed to the wrong panel, delayed or rejected clinical feeds, missing diagnosis or procedure mappings, care gaps opening or closing incorrectly, outreach queues not refreshing, measure value sets using the wrong version, user permissions hiding a population, and dashboard totals disagreeing with patient-level detail.

ibm-senior-data-scientist-09

Created and delivered Phytel configuration training covering provider activation, system changes, support procedures, and the clinical-data workflows underlying proactive population-health management.

S Explorys developers were being asked to support integrations and workflows that crossed into Phytel, but they had not worked with Phytel's provider, patient, care-gap, and outreach data before.

T I needed to give those developers enough domain and data-model context to diagnose cross-product issues safely.

A I created training that mapped familiar Explorys concepts to Phytel entities and workflows, traced provider activation and clinical data through care-gap calculation and outreach, demonstrated common configuration failures, and used realistic support cases as exercises.

R Explorys developers could follow Phytel data end to end, distinguish configuration defects from ingestion or application defects, and collaborate more effectively on cross-product support.

ibm-senior-data-scientist-11

Developed and delivered peer-led Azure instruction through the REACH initiative, translating cloud architecture, storage, data integration, security, and analytics concepts into practical guidance for technical colleagues.

S Colleagues familiar with the legacy platform needed a practical way to understand Azure, not a catalog of services.

T Through REACH, I needed to connect cloud concepts to work they already performed.

A I designed instruction around architecture, storage, integration, security, and analytics, mapped legacy patterns to Azure equivalents, used migration exercises, and revised material from participant questions.

R Colleagues gained a working mental model for evaluating Azure designs and could participate more effectively in migration discussions.

N **Notes / audit prompts**

Likely topics included Azure landing-zone and subscription structure; Blob Storage and ADLS Gen2; Azure Data Factory pipelines, triggers, parameters, and integration runtimes; Synapse SQL and Spark workloads; managed identities, Key Vault, RBAC, and private networking; monitoring with Azure Monitor and Log Analytics; schema drift, partitioning, and migration reconciliation; cost controls; and CI/CD through Azure DevOps.

ibm-senior-data-scientist-10

Performed postmigration validation for an enterprise Jira transition, reconciling projects, issues, workflows, custom fields, permissions, histories, and relationships and documenting discrepancies for remediation before cutover.

S A Jira migration could appear complete while losing workflow states, fields, permissions, history, or links that teams relied on.

T I needed to validate functional equivalence before cutover.

A I built source-to-target inventories and counts for projects, issues, workflows, fields, permissions, histories, and relationships; tested complex cases; categorized findings by severity; and documented reproducible defects for remediation and retest.

R The team had acceptance evidence and a clear defect list, reducing the risk of missing access or broken work history after migration.

IBM – Senior Data Architect

Dates: Dec. 2020 - Nov. 2021

Directed data-onboarding architecture and governance for large-scale clinical platforms, combining metadata-driven integration, deidentification, lineage, validation, and applied machine learning.

`ibm-senior-data-architect-02`

Served as a subject-matter expert to project data architects and delivery, analytics, and integration teams; directed configuration of the data-onboarding platform and supporting infrastructure.

S Project architects and delivery, analytics, and integration teams made interconnected configuration decisions across the onboarding platform.

T I needed to provide a consistent technical point of view and direct configurations toward reusable patterns.

A I reviewed source designs, clarified grains and mappings, identified infrastructure dependencies, resolved cross-team questions, and required validation evidence before configurations advanced.

R Teams made faster decisions with fewer contradictory assumptions, and onboarding configurations aligned more consistently with platform standards.

`ibm-senior-data-architect-09`

Replaced static mappings with metadata-driven instruction sets across approximately 200 source integrations, standardizing EDI batch schemas while enabling Python/Jupyter lineage tracing and end-to-end validation gates.

S Approximately 200 integrations were mapped through source-controlled Excel workbooks. The files mixed requirements, field mappings, transformations, and comments in layouts that were difficult to validate automatically or execute consistently.

T I needed to preserve the subject-matter knowledge in those workbooks while separating declarative integration instructions from reusable execution code.

A I defined versioned, machine-readable instruction sets—normalized CSV or JSON manifests loaded into metadata tables—with one record per target field and attributes for source path, datatype, cardinality, default or null handling, lookup, transformation expression, terminology mapping, dependency, and validation rule. Python converted approved workbook content into the instruction schema, Spark/Hadoop jobs interpreted it against standardized EDI batches, and Jupyter notebooks traced each target field back through the applied instructions and validation results.

R Teams retained governed, reviewable mapping content while gaining reusable execution, automated checks, and consistent lineage across roughly 200 integrations.

`ibm-senior-data-architect-07`

Architected Spark and Hadoop pipelines integrating more than 100 billion structured and semi-structured clinical records, supporting four EPM applications, 30+ risk models, and hundreds of measures and registries.

S Four EPM applications, more than 30 risk models, and hundreds of measures depended on over 100 billion clinical records from heterogeneous sources.

T I needed to design pipelines that could scale without losing lineage or consistent analytical meaning.

A I defined layered Spark and Hadoop processing, partition and schema strategies, metadata and identity rules, and validation gates for structured and semi-structured inputs; I then tuned heavy stages using observed execution behavior.

R The platform supplied reusable, governed datasets at the scale required by applications, models, measures, and registries.

N **Notes / audit prompts**

The 30+ models most likely included combinations of:

- HCC/RAF and other prospective cost-risk models;
- LACE, Charlson, and Elixhauser risk or comorbidity scores;
- 30-day readmission, inpatient-admission, and emergency-department utilization risk;
- mortality, length-of-stay, and avoidable-utilization models;
- disease-specific deterioration or complication models for conditions such as heart failure, COPD, diabetes, and sepsis; and
- care-gap, rising-risk, and high-cost claimant stratification.

ibm-senior-data-architect-08

Led a HIPAA patient-deidentification project drawing from more than 100 data sources for content analytics and measures.

S Content analytics and measures required data from more than 100 sources, each with different identifiers and potential protected-health-information exposure.

T I needed to lead a common deidentification approach that protected privacy while preserving analytical relationships.

A I inventoried sensitive fields, translated policy into source-specific transformations, standardized pseudonymous identities, tested for residual identifiers and broken linkages, and coordinated exceptions with governance and engineering.

R The project delivered a consistent deidentified foundation for downstream analytics with documented controls and traceability.

N **Notes / audit prompts**

A likely HDFS implementation began by choosing the approved HIPAA path—Safe Harbor or a documented Expert Determination—then inventorying identifiers in structured and free-text fields. Spark, Hive, or MapReduce transformations may have removed direct identifiers; tokenized patient, provider, account, and encounter keys; generalized or suppressed geography and ages; shifted dates consistently within each patient; and redacted recognizable identifiers from notes. Deterministic crosswalks would have been stored separately in restricted HDFS locations, while deidentified Hive views preserved clinical relationships for measures. Validation likely combined pattern scans, referential-integrity checks, uniqueness or reidentification-risk review, access-control tests, and lineage evidence. Audit point: confirm whether the governing method was Safe Harbor, Expert Determination, or a limited-data-set policy rather than describing all three interchangeably.

ibm-senior-data-architect-03

Led a cross-functional team as the clinical data analyst for the 2020/2021 measure-updates project.

S The 2020/2021 HEDIS release changed measure specifications, value sets, look-back windows, exclusions, and continuous-enrollment rules used across products. Inconsistent interpretation could change eligible populations and reported rates.

T As the clinical data analyst leading a cross-functional team, I needed to translate the annual HEDIS changes into implementable data logic and validated outputs.

A I compared specifications year over year, decomposed changes into denominator, numerator, exclusion, enrollment, and event-date rules, mapped new value sets to clinical fields, assigned work across engineering and clinical reviewers, and built test cohorts for boundary dates, enrollment gaps, exclusions, and numerator compliance. I reconciled old and new rates and explained expected population movement.

R The team delivered the HEDIS update with a shared interpretation of each changed rule and traceable evidence for resulting rate and population differences.

ibm-senior-data-architect-01

Served as a senior member of Data Onboarding & Governance, integrating healthcare data, enriching metadata, and configuring data models, analytics, reports, and visualizations.

S New healthcare datasets had to become usable across models, analytics, reports, and visualizations while meeting governance expectations.

T As a senior Data Onboarding and Governance member, I needed to connect technical ingestion with downstream meaning.

A I profiled sources, mapped entities and metadata, configured data models and analytical outputs, documented lineage and constraints, and validated results with delivery and analytics teams.

R Onboarded data arrived with a clearer contract and provenance, giving downstream teams a more reliable foundation for reporting and analysis.

ibm-senior-data-architect-10

Served in an advisory capacity for development of a new data platform.

S A new data platform was being designed while teams still depended on legacy integration patterns and healthcare-specific workflows.

T In an advisory role, I needed to bring practical delivery constraints into architecture decisions without owning the entire build.

A I reviewed proposals, translated legacy lessons into requirements, identified migration and governance risks, challenged assumptions with representative workloads, and recommended validation and lineage capabilities.

R The platform team could evaluate design choices against real operating needs and address important compatibility and control gaps earlier.

ibm-senior-data-architect-05

Developed a topic-modeling project using nearly six million semi-structured provider notes.

S Nearly six million provider notes contained useful themes that were difficult to discover through manual review or fixed keyword searches.

T As the sole contributor, I needed to determine whether topic modeling could organize the corpus for analyst exploration.

A I built the full NLP workflow: deidentified and normalized text, removed boilerplate and low-information terms, created bag-of-words and TF-IDF representations, and compared Latent Dirichlet Allocation and non-negative matrix factorization across topic counts. I evaluated coherence, stability, top terms, and representative notes, then used dimensionality reduction and clustering views to inspect overlap and worked with domain reviewers to label useful topics.

R My solo project produced an exploratory map of recurring clinical themes and documented where LDA and NMF could assist, but not replace, expert review.

ibm-senior-data-architect-06

Developed a surgical-scheduling optimizer and CNN/RNN case-card prediction proof of concept that was being evaluated as a potential new offering.

S Surgical teams faced schedule inefficiency and uncertainty about the supplies or case-card content needed for upcoming procedures.

T On a three-person team, I needed to test whether optimization and sequence models could improve planning enough to justify a product offering.

A We divided the work across clinical-data preparation, scheduling optimization, and case-card prediction. I helped define constraints and objectives, engineered historical procedure and item sequences, implemented and evaluated CNN/RNN prediction experiments, and integrated results with the optimizer for stakeholder review.

R The three-person team produced a working proof of concept with performance evidence and implementation risks that product leaders could use to evaluate a potential offering.

ibm-senior-data-architect-04

Created an exploratory machine-learning COVID-19 prototype to surface hospitalization-risk signals for retrospective analyst review.

S Early in the COVID-19 response, analysts needed a rapid way to explore which historical patient characteristics were associated with hospitalization.

T On a three-person team, I helped build a retrospective prototype while keeping it clearly separate from a clinical decision system.

A We defined the cohort, index date, and hospitalization outcome; engineered demographic, comorbidity, utilization, and laboratory features; and compared regularized logistic regression, random forest, and gradient-boosted tree models. I contributed feature preparation, cross-validation, precision-recall and ROC analysis, calibration review, and feature-importance interpretation while documenting bias, missingness, and temporal instability.

R The prototype gave analysts an interpretable comparison of candidate risk signals and a documented basis for deciding what merited further study, without presenting exploratory output as deployable clinical guidance.

`ibm-senior-data-architect-11`

Closed project-documentation gaps by organizing material from more than 100 past projects.

S Useful decisions, mappings, and troubleshooting knowledge from more than 100 projects were scattered across inconsistent locations and formats.

T I needed to preserve what was reusable and make it discoverable.

A I inventoried available material, grouped it by workflow and subject, removed duplication, identified missing context, established naming and navigation conventions, and highlighted patterns and lessons that applied across clients.

R Teams gained a more usable project knowledge base, reduced repeated investigation, and had better starting material for onboarding and future delivery.

IBM – Data Architect

Dates: Nov. 2017 - Dec. 2020

Led healthcare-data onboarding and validation across complex multi-EHR integrations, strengthening data-sharing controls, patient-matching quality, and repeatable client delivery.

`ibm-data-architect-02`

Architected data onboarding for the Central New York Care Collaborative on IBM Watson Health's Explorys platform, integrating 75+ EHR, HIE, and other source systems across six counties; implemented provider-scoped patient filtering to enforce data-sharing rules and coordinated delivery across engineering teams, earning promotion to Senior Data Architect.

S The Central New York Care Collaborative was building a regional population-health platform for more than 2,000 providers across six counties, with data arriving from 75+ EHR, HIE, and other source systems. Data-sharing rules required patient information to remain within valid provider relationships.

T As the data architect, I needed to establish a scalable onboarding architecture that normalized heterogeneous sources, preserved traceability, and enforced provider-level sharing boundaries.

A I defined source contracts and canonical mapping patterns, modeled organization-provider-patient relationships, placed provider-scoped filtering early in the pipeline, built leakage, completeness, and reconciliation tests, documented reusable implementation patterns, and coordinated edge cases and delivery across engineering teams.

R The architecture provided a consistent, auditable pattern for onboarding diverse healthcare sources while enforcing data-sharing constraints. My technical leadership and architectural ownership on the project earned me a promotion to Senior Data Architect.

`ibm-data-architect-01`

Designed SQL validation frameworks and client-review gates across approximately 100 integrations, reducing Jira-tracked validation rework by 25% and mapping rework by 45%.

S Client integrations repeatedly reached review with preventable mapping and validation defects, creating Jira rework late in delivery.

T I needed to move consistent checks ahead of client review across approximately 100 integrations.

A I built reusable SQL validation patterns for counts, keys, required fields, mappings, and business rules, defined review gates and evidence, and fed recurring defects back into templates and guidance.

R Jira- tracked validation rework fell 25% and mapping rework fell 45%, giving teams faster, more predictable client review cycles.

`ibm-data-architect-03`

Built the data-onboarding platform and associated infrastructure, remediated UAT data issues, and reported on data health during onboarding.

S New clients needed a repeatable route from raw source delivery to usable platform data, while UAT defects and unclear data health slowed acceptance.

T I needed to establish the onboarding foundation and make readiness visible.

A I configured platform and infrastructure components, defined source-to-target stages and checks, triaged UAT examples to the responsible mapping or transformation, and summarized completeness, quality, and open issues in data-health reporting.

R Stakeholders could see what was ready, what was blocked, and why, and the team had a more repeatable process for moving integrations through UAT.

N **Notes / audit prompts**

On HDFS, this likely meant landing immutable client extracts in secured raw directories; registering file, partition, and schema metadata in Hive; using Pig, Spark, or MapReduce jobs to standardize records into staged and curated zones; and maintaining project-level configuration for source layouts, mappings, and run parameters. UAT defects would be reproduced against known patient or encounter examples, traced through raw, staged, normalized, and application-ready outputs, and rerun after correction. Data-health reporting likely summarized delivery freshness, file and row counts, rejected records, missing required fields, duplicate keys, code-mapping coverage, referential integrity, and unresolved defects by source and processing stage.

ibm-data-architect-04

Earned AI certification and peer-reviewed patient-matching algorithm tuning across the project portfolio.

S Patient-matching changes could reduce duplicates but also create harmful false matches across many client projects.

T I needed to apply stronger AI knowledge and an independent review discipline to tuning decisions.

A After completing AI certification, I reviewed proposed features, thresholds, training or test cohorts, and error examples; compared precision and recall tradeoffs; and required project teams to document why a change was appropriate for their data.

R Tuning decisions received a consistent peer review and clearer evidence before being applied across the portfolio.

ibm-data-architect-08

Received a GDP bonus for improving revenue per SaaS client by at least 1.34%.

S SaaS client delivery included recurring effort that limited the team's capacity to provide additional billable value.

T I needed to improve the economics of each client without reducing service quality.

A I reviewed where onboarding and support effort was being consumed, standardized repeatable validation and configuration work, and helped redirect recovered capacity toward higher-value client services and timely delivery.

R Revenue per SaaS client improved by at least 1.34%, and the contribution was recognized through a GDP bonus.

ibm-data-architect-07

Received the Lab Services Award for coordinating internal project teams to deliver client needs on time.

S The Central New York Care Collaborative engagement spanned 75+ EHR, HIE, and other source systems and depended on IBM onboarding, platform, engineering, and delivery teams with separate priorities. Provider-scoped data-sharing rules made handoffs especially sensitive.

T I needed to coordinate the internal work so CNYCC received the required integration behavior and evidence on schedule.

A I translated the CNYCC requirements into shared milestones, mapped owners and dependencies, coordinated provider-attribution and filtering changes, surfaced blockers and edge cases early, and kept engineering, validation, and delivery teams aligned through implementation and client review.

R The teams met the CNYCC delivery commitment with auditable provider-scoped controls, and IBM recognized my cross-team coordination and client impact with the Lab Services Award.

ibm-data-architect-05

Curated 25 pages of AI-workflow documentation and mentored new hires in the AI-tuning process.

S The AI-tuning workflow depended on aging legacy documentation, and significant steps, decision criteria, and failure modes were either incomplete or undocumented. New hires therefore relied too heavily on oral guidance.

T I needed to repair the existing material, close the major gaps, and make the workflow teachable without oversimplifying patient-matching risk.

A I reviewed the legacy pages against the actual process, corrected obsolete instructions and screenshots, reorganized fragmented content, and wrote new sections covering data preparation, parameter effects, evaluation measures, false-match and missed-match review, approval points, rollback, and common failure modes. I curated the result into 25 pages and mentored new hires through representative tuning cases.

R The team gained a current end-to-end reference instead of relying on tribal knowledge, and new hires could contribute with greater consistency and awareness of matching tradeoffs.

ibm-data-architect-06

Spearheaded a new-hire mentoring and onboarding program from 2016 to 2018, standardizing team practices, systems, and technical workflows to prepare colleagues for independent, billable client delivery.

S The existing new-hire plan stretched across five loosely structured weeks and repeatedly pulled senior architects and developers away from client delivery to reteach the same material.

T From 2016 to 2018, I needed to shorten ramp-up, improve consistency, and return senior staff to day-to-day project work.

A I redesigned the five-week plan as an intensive two-week curriculum, took primary responsibility for instruction, sequenced platform, healthcare-data, mapping, validation, and delivery exercises, and used practical checkpoints to confirm readiness. I then added a structured three-week mentorship phase in which each new hire applied the material to representative integration work with targeted review rather than continuous senior-led training.

R New employees reached productive, billable work faster, senior architects and developers recovered time for client and production issues, and the team reduced the labor cost of onboarding while preserving three weeks of supported practice.

ibm-data-architect-09

Served on tiger teams supporting the HDFS-to-Azure data-platform upgrade and participated on the data-governance team.

S The HDFS-to-Azure upgrade affected shared clinical data, downstream applications, and governance controls across multiple teams.

T I was asked to help resolve cross-cutting migration risks and preserve trusted data behavior.

A On tiger teams, I traced source-to-target dependencies, analyzed validation failures, documented platform constraints, and brought findings into governance discussions about lineage, access, retention, and quality. I then retested changes against affected workflows.

R The program could make architecture and governance decisions from concrete data evidence, and high-risk issues were surfaced before they reached migration sign-off.

ibm-data-architect-10

Played a central role in the REACH cross-team training initiative, teaching SQL fundamentals to non-technical colleagues and ETL, architecture, data-science, and predictive-modeling practices to technical colleagues.

S IBM colleagues needed different levels of data fluency, from basic SQL interpretation to practical ETL, architecture, data-science, and modeling skills.

T I needed to make cross-team instruction useful to both nontechnical and technical audiences.

A Through REACH, I separated the curriculum by audience, built examples around familiar healthcare-data problems, combined explanation with hands-on exercises, and adjusted later sessions from participant questions and observed gaps.

R Colleagues gained a shared vocabulary and practical techniques they could apply in project discussions, analysis, and delivery rather than receiving one generic training course.

IBM – Software Developer, Data Platform

Dates: Sep. 2016 - Nov. 2017

Developed healthcare data-platform tooling and training programs that automated HL7 ingestion, reduced vendor costs, and accelerated knowledge transfer.

`ibm-software-developer-01`

Engineered an HL7 v2/v3 parsing pipeline in Pig for approximately 30 internal users, replacing a Mirth-based workflow and eliminating $30,000 in annual vendor costs.

S Approximately 30 internal users depended on a Mirth-based process to parse HL7 v2 and v3 data, creating licensing cost and limiting control over changes.

T I needed to replace the workflow without disrupting existing consumers.

A I documented supported message structures, built a Pig pipeline to parse and normalize segments and fields, compared outputs against the Mirth results, added malformed-message handling, and supported a controlled transition with user guidance.

R Internal users retained the required HL7 data while IBM eliminated approximately $30,000 in annual vendor costs.

`ibm-software-developer-05`

Received a GDP bonus for improving revenue per SaaS client by at least 1.17%.

S Repeated integration and support effort constrained how much value the team could deliver per SaaS client.

T I needed to reduce avoidable effort and expand capacity for billable work.

A I automated a recurring technical workflow, strengthened documentation and training, and standardized delivery steps so specialists spent less time on manual setup and repeated troubleshooting.

R The recovered capacity contributed to at least a 1.17% improvement in revenue per SaaS client and was recognized through a GDP bonus.

`ibm-software-developer-04`

Received the Eminence and Excellence Award in 2017 for an analytics project.

S A project team had a large operational dataset but lacked a concise analytical view of the patterns driving a client problem.

T I needed to turn the data into a defensible analysis that stakeholders could act on.

A I clarified the business question, prepared and validated the analytical dataset, tested alternative explanations, presented the important patterns with their limits, and packaged the method so the analysis could be repeated.

R The work informed the project team's decisions and was recognized with IBM's 2017 Eminence and Excellence Award.

N **Notes / audit prompts**

A plausible client problem was an apparent drop in the number of patients reaching downstream quality and population-health cohorts even though inbound encounter volume remained stable. The analysis may have reconciled source files, rejected records, identity matches, provider attribution, and cohort eligibility by facility and load date, revealing that a small set of changed source codes or provider mappings caused a disproportionate share of exclusions. The deliverable would have turned an ambiguous "missing patients" complaint into a ranked remediation plan and repeatable monitoring. Audit point: confirm the client, affected workflow, and whether the award recognized this type of cohort-loss analysis or a different analytics project.

`ibm-software-developer-02`

Rewrote and reorganized project documentation to streamline knowledge transfer.

S More than 200 client integrations each required project-specific knowledge, but documentation was inconsistent and often written after delivery, making support and handoff depend on the original developer.

T I needed to make documentation a maintained part of the ETL lifecycle for every integration rather than an optional closeout activity.

A I created a standard project structure covering source inventory, mappings, transformation rules, job configuration, validation evidence, deployment, operations, troubleshooting, and client-specific exceptions. I wrote and maintained the documentation for each integration I touched, embedded review and update checkpoints into the team's ETL process, and promoted the structure across the 200+ integration portfolio.

R Documentation became a delivery artifact that stayed aligned with implementation, improving knowledge transfer, production support, and reuse across client projects.

`ibm-software-developer-03`

Redeveloped the employee education program to make new-hire training more efficient and repeatable.

S New-hire education varied by mentor and project assignment, so important platform concepts were taught inconsistently.

T I needed to turn training into a repeatable program that still prepared people for real delivery.

A I organized role-based modules, sequenced foundational and advanced topics, created hands-on exercises and reference material, added checkpoints and a representative capstone task, and incorporated learner and mentor feedback.

R New hires received a clearer progression, mentors could reuse the same structure, and the team spent less time rebuilding training for each cohort.

`ibm-software-developer-06`

Participated in internal coding jams, hackathons, and cross-team delivery initiatives.

S Teams working on different products often encountered similar ingestion, analytics, and workflow problems but had few opportunities to experiment together outside committed delivery.

T I wanted to test ideas quickly and build connections across specialties.

A I joined coding jams, hackathons, and cross-team initiatives, helped narrow broad ideas into demonstrable use cases, contributed data preparation and analytical code, and presented results and limitations to colleagues.

R The events produced reusable technical approaches and working relationships that informed later project delivery, even when a prototype itself did not become a production product.

IBM – Data Scientist

Dates: Nov. 2015 - Sep. 2016

Directed client data-integration delivery and built reusable extraction and QA practices that reduced implementation effort and standardized onboarding across dozens of healthcare projects.

ibm-data-scientist-01

Directed technical delivery across 36 client data integrations, coordinating 12 remote contractors and establishing code-review, QA, and issue-resolution standards.

S Thirty-six client integrations were being delivered with 12 remote contractors, making quality, communication, and issue ownership inconsistent.

T I needed to coordinate parallel delivery without becoming the only person who could resolve every problem.

A I clarified work packages and owners, established code-review and QA expectations, introduced a common issue- resolution path, reviewed high-risk mappings, and used recurring defects to improve standards.

R Contractors could deliver against shared expectations, issues reached the right owner faster, and integrations progressed with more consistent technical quality.

ibm-data-scientist-02

Served as principal integration specialist on more than 60 projects for 20 clients across the broader 2015–2018 data-ingestion period.

S More than 60 projects for 20 clients brought different systems, healthcare workflows, and delivery schedules, but many integration problems repeated.

T As principal specialist, I needed to solve the immediate client issue while building reusable expertise.

A I led discovery, profiled source data, defined mappings and acceptance checks, diagnosed ingestion failures, and turned recurring patterns into tools, documentation, and review guidance.

R Teams could apply proven approaches across clients, reducing reinvention while retaining the source-specific analysis each integration required.

ibm-data-scientist-04

Built a Python, Ruby, and SQL extraction tool that eliminated unnecessary story points and reduced average engineering hours per client integration.

S Engineers spent repeated effort inspecting source systems and assembling the same metadata and extracts before integration work could begin.

T I needed to remove avoidable manual steps from each client onboarding.

A I built a Python, Ruby, and SQL tool that connected to supported sources, extracted required structures and sample information, normalized the output, and produced artifacts developers could use for mapping and estimation.

R The tool eliminated unnecessary story points and lowered average engineering hours per integration while making discovery output more consistent.

ibm-data-scientist-03

Authored 125+ technical knowledge-base articles, approximately 80% of the team's published content, to standardize integration development and troubleshooting practices.

S Integration developers repeatedly asked the same setup, mapping, and troubleshooting questions because answers were scattered in tickets and personal notes.

T I needed to make operational knowledge searchable and reusable.

A I identified recurring questions, wrote articles with context, prerequisites, steps, examples, expected results, and failure paths, linked related topics, and updated guidance when platform behavior changed.

R I produced more than 125 articles—about 80% of the team's published content—giving the organization a consistent reference for development and incident resolution.

ibm-data-scientist-05

Eliminated an average of three weeks from employee onboarding.

S New integration specialists spent too long locating documentation, gaining system access, and learning the sequence from source discovery to delivery.

T I needed to shorten ramp-up without skipping essential practice.

A I reorganized onboarding around a defined learning path, consolidated reference material, introduced guided exercises and a representative integration, paired new hires with reviewers, and used checkpoints to target remaining gaps.

R Employees reached productive project work an average of three weeks sooner, while managers retained a structured way to assess readiness.

Explorys, an IBM Company – Data Scientist

Dates: Jun. 2015 - Nov. 2015
Technologies: Hadoop/HDFS, Ruby, Pig, React, Node.js, JavaScript, SQL, Oracle, Jenkins

Joined Explorys during a period of rapid client expansion and its transition into IBM Watson Health. Helped scale healthcare data onboarding across more than 10 client warehouses by leading source discovery and integration setup, diagnosing Hadoop and Oracle ingestion failures, and introducing scheduling, recovery, and blackout controls for production data feeds. Contract via Ranstad USA.

`explorys-02`

Identified and extracted data from heterogeneous client systems; led technical requirements gathering, discovery, documentation, and implementation; and coordinated acquisition, installation, and configuration of VPN hardware supporting client integrations.

S Each new healthcare client exposed different databases, schemas, network constraints, and operational contacts, so onboarding could not begin with a standard extract alone.

T I needed to carry integrations from technical discovery through a secure, validated initial data flow.

A I led source and requirements sessions, inventoried tables and refresh behavior, wrote extraction and mapping documentation, implemented and validated the data pull, and coordinated procurement, installation, configuration, and connectivity testing for required VPN hardware.

R Client teams and Explorys had a shared integration contract, secure connectivity, and a repeatable path from heterogeneous source data to the onboarding platform.

`explorys-01`

Diagnosed Hadoop and Oracle production-ingestion failures and implemented Jenkins-backed scheduling and recovery controls across 10+ client data warehouses.

S Rapid client growth left more than 10 data warehouses dependent on ingestion jobs that could fail at the Oracle source, during transfer, or in Hadoop, with inconsistent recovery steps.

T I needed to make failures diagnosable and restore feeds without rerunning more work than necessary.

A I traced failed loads through source queries, transfer logs, HDFS landing data, and downstream jobs; documented recurring causes; and implemented Jenkins scheduling, dependency, restart, and notification controls around the ingestion workflow.

R Production support gained a consistent way to identify the failed stage and recover client feeds, reducing reliance on ad hoc manual restarts.

`explorys-03`

Developed a cron-based control for defining blackout periods around client data pulls.

S Scheduled data pulls could overlap client maintenance, backups, or other restricted periods, creating avoidable source contention and failed loads.

T I needed a lightweight way to respect client-specific blackout windows without manually disabling and re-enabling each integration.

A I built a cron-based control that read configured blackout periods before starting a pull, skipped or deferred ineligible runs, logged the decision, and resumed the normal schedule afterward. I tested boundary times and representative client schedules.

R The platform could honor operating constraints consistently, reducing preventable collisions while preserving an auditable schedule for production support.

AmTrust Financial Services – Software Engineer

Dates: Mar. 2014 - Jun. 2015
Technologies: VB.NET, ASP.NET, C#, Web APIs, CSS, JavaScript, HTML5, SQL, PL/SQL, LINQ, R

Brought in to develop agent facing commercial fire insurance quoting and policy generation software. Grew into the principal developer for a new product offering, translating ISO rating rules into more than 65 web forms, APIs, and SQL services while strengthening testing, performance, architecture, and coordination between business analysts and developers.

amtrust-02

Implemented a new line of business in VB.NET and Microsoft SQL Server, served as principal developer for a new product offering, and contributed to two major software-architecture initiatives in 2015.

S AmTrust wanted to introduce a new commercial-insurance offering whose rating rules, forms, and policy workflow did not fit the existing product unchanged.

T As principal developer, I needed to turn the product concept into a maintainable VB.NET and SQL Server implementation.

A I decomposed the line of business into shared and product-specific rules, designed the data and service changes, coordinated form and API delivery, reviewed implementation decisions, and carried lessons into two broader architecture initiatives.

R The new offering gained a working technical foundation while its design contributed to more reusable platform architecture in 2015.

amtrust-07

Developed more than 65 ISO rating-based web forms and analyzed page timings in R to identify optimization targets and improve application performance.

S A new commercial product required more than 65 web forms derived from ISO rating rules, and the resulting pages varied in complexity and response time.

T I needed to deliver the rule-driven forms while identifying where users experienced avoidable delay.

A I translated rating requirements into reusable form and validation patterns, implemented the supporting services and data calls, captured page-timing observations, and used R to compare distributions and locate slow outliers before profiling their rendering or query paths.

R The product received broad ISO-based form coverage and a data-driven set of performance improvements rather than ad hoc optimization.

amtrust-09

Supported the technical integration of acquired businesses into the commercial policy platform.

S Acquired businesses brought policy products, data, users, and operating workflows that differed from AmTrust's commercial platform.

T I needed to help incorporate those businesses without interrupting active policy work.

A I inventoried source applications and dependencies, mapped products and policy data to platform concepts, identified functional and technical gaps, supported interface and migration changes, and validated representative transactions through a phased integration plan.

R The acquired operations gained a defined route onto the commercial-policy platform while material differences and continuity risks were addressed before broader rollout.

amtrust-04

Served as the working liaison between business analysts and developers, translating requirements, reporting progress, and helping prioritize delivery needs.

S Business analysts understood insurance rules while developers understood the application, but ambiguous translation between them created scope and acceptance risk.

T I needed to maintain a shared understanding throughout delivery.

A I walked analysts through examples and edge cases, translated decisions into technical behavior and acceptance criteria, explained constraints to both groups, reported progress and blockers, and helped sequence work by customer and implementation impact.

R Questions were resolved earlier, developers worked from clearer requirements, and stakeholders had a more reliable view of what would be delivered and when.

amtrust-03

Evaluated the advantages, risks, and implementation costs of requested features and recommended maintainable solutions grounded in object-oriented design principles.

S Feature requests often arrived with an expected interface change but hidden consequences for rating logic, data, integrations, and future maintenance.

T I needed to help stakeholders choose solutions based on total impact, not only initial coding effort.

A I clarified the business outcome, traced affected components, estimated implementation and support costs, compared alternatives and failure risks, and recommended designs that used clear responsibilities, encapsulation, and reusable abstractions.

R Product decisions had an explicit technical rationale, and the team avoided short-term changes that would have created disproportionate complexity later.

amtrust-01

Designed, developed, tested, and maintained full-stack commercial-insurance applications spanning VB.NET and ASP.NET interfaces, multi-tier components, Web APIs, and SQL data services.

S Commercial-insurance quoting and policy generation crossed web forms, application services, rating logic, APIs, and database procedures, so a change in one tier could create defects elsewhere.

T I needed to deliver features end to end while preserving consistent behavior across the application.

A I traced requirements through the VB.NET and ASP.NET interfaces, multi-tier components, Web APIs, and SQL services; implemented the coordinated changes; added unit and integration coverage; and validated representative quote and policy scenarios before release.

R The team could deliver commercial-insurance capabilities through a controlled full-stack process with clearer traceability and fewer handoff gaps.

amtrust-06

Created a unit- and integration-test suite for multi-tier MVC components, improving regression coverage and establishing stronger team testing practices.

S Changes to multi-tier MVC components were difficult to verify without repeatedly exercising entire quote and policy workflows by hand.

T I needed to create a safety net that covered both isolated logic and important tier interactions.

A I separated testable responsibilities, wrote unit tests for rating and application behavior, built integration tests across controllers, services, and data access, created representative fixtures, and used failures to refine team review expectations.

R Regression coverage improved, developers received faster feedback on changes, and automated testing became a stronger part of normal delivery practice.

amtrust-05

Coordinated developer schedules and troubleshot production problems across multiple application environments.

S Multiple application environments and concurrent releases created coordination risk, while production issues could involve code, configuration, data, or deployment differences.

T I needed to keep planned work moving while restoring service safely when incidents occurred.

A I coordinated developer assignments and dependencies, compared behavior across environments, reviewed logs and database state, reproduced failures with representative cases, organized the appropriate fix and validation, and documented follow-up actions.

R Production problems reached the right owner faster, releases had clearer sequencing, and fixes were validated across environments before closure.

amtrust-08

Refactored and maintained the existing VB.NET insurance-rating codebase to improve readability, performance, and long-term maintainability.

S The existing VB.NET rating code had accumulated intertwined rules, duplicated logic, and performance-sensitive paths that made changes risky.

T I needed to improve maintainability without changing valid insurance outcomes.

A I identified duplication and complex dependencies, separated calculation, presentation, and data responsibilities, introduced clearer interfaces and naming, expanded regression tests around representative ratings, and profiled the most expensive paths before optimizing them.

R The code became easier to review and extend, high-use paths performed more efficiently, and future rating changes could be made with a stronger regression safety net.

PPG Industries – Color Scientist

Dates: Apr. 2012 - Aug. 2013
Technologies: VB.NET, C#, SQL, C++, FORTRAN, JMP, spectrophotometry, numerical methods

Joined PPG as a color scientist and software specialist supporting the modernization of legacy automotive color-matching technology. Applied spectroscopy, radiative-transfer physics, statistical modeling, and numerical methods to improve matching and measurement consistency, then became a global liaison helping translate that work into software, standardized workflows, training, and operational adoption.

ppg-06

Served as technical voice for color data-management software, supporting its global development, launch, training, adoption, and application.

S A global color data-management product had to serve laboratories, operations, and customer-facing teams with different terminology and workflows.

T I needed to represent the underlying color science throughout product development and help users apply the software correctly after launch.

A I translated scientific requirements into software behavior, reviewed prototypes and test cases, resolved questions from regional teams, developed technical training and examples, and incorporated launch feedback into guidance and enhancements.

R The product launched with a more consistent scientific model and users had a technical resource for training, adoption, and difficult application questions.

ppg-08

Served as the global liaison among Color, Technical, Operations, and IT teams, translating color-science requirements into consistent systems, workflows, and communications.

S Regional Color, Technical, Operations, and IT groups used different vocabulary, priorities, and local practices, creating inconsistent interpretations of the same color-science requirement.

T I needed to establish a shared understanding that could be implemented globally without ignoring local constraints.

A I facilitated cross-functional discussions, documented the scientific intent and decision rules, mapped regional workflows and system impacts, resolved conflicting assumptions, and circulated examples and decisions in a common format.

R Teams worked from more consistent requirements and could coordinate system changes, operating practices, and communications with fewer regional surprises.

ppg-12

Championed a more science-centered organizational approach to color by introducing analytical methods, shared standards, and evidence-based decision-making.

S Color decisions were sometimes driven by local custom or individual experience, making it difficult to compare results and scale improvements across teams.

T I wanted to strengthen the role of reproducible evidence without discounting the expertise of experienced practitioners.

A I introduced common measurement and analysis methods, demonstrated them on active business questions, encouraged teams to document assumptions and acceptance criteria, and used shared standards to connect expert judgment with quantitative evidence.

R Teams gained a more consistent language for evaluating color performance and a stronger basis for repeating, reviewing, and communicating technical decisions.

ppg-11

Developed statistical models from established theory, experimental data, and novel approaches to improve color measurement, matching, and process decisions.

S Established color theory explained much of the measurement process, but real production data also contained instrument, material, and process variation that theory alone did not capture.

T I needed to build models that remained scientifically grounded while being useful for operational decisions.

A I combined theoretical predictors with designed experiments and historical observations, explored additional features and interactions, validated performance on held-out or repeated measurements, and translated model limitations into decision thresholds.

R The models provided more actionable estimates for measurement quality, match selection, and process control while making uncertainty explicit.

ppg-10

Evaluated color processes, instrumentation, and data flows to identify technical and operational risks and develop practical mitigation strategies.

S Color results could drift because of instrument condition, sample preparation, environmental effects, data transformations, or inconsistent operating steps.

T I needed to identify where those risks entered the workflow and recommend controls that operations could realistically sustain.

A I mapped the process from sample preparation through measurement, data transfer, and decision-making; reviewed failure modes and available evidence; ranked risks by impact and detectability; and proposed calibration, validation, monitoring, and escalation controls.

R Stakeholders gained a practical risk view of the end-to-end color process and a prioritized set of mitigations rather than isolated equipment fixes.

ppg-07

Managed a portfolio of color-related intellectual property and supported the translation of research into operational tools and practices.

S Color research generated algorithms, methods, and experimental findings whose value could be lost if ownership, novelty, and practical application were not tracked.

T I needed to help manage that portfolio while moving useful ideas from research into operations.

A I cataloged candidate inventions and supporting evidence, coordinated technical review with research and IP stakeholders, distinguished protectable concepts from implementation know-how, and helped translate selected methods into software requirements, procedures, and training.

R PPG had clearer visibility into its color-related intellectual assets and a more direct path from promising research to repeatable operational use.

ppg-05

Led cross-functional color-science and marketing initiatives from concept through on-time, in-full delivery for the Automotive Refinish business.

S Automotive Refinish initiatives often combined technical color claims, customer-facing materials, software behavior, and launch deadlines owned by different functions.

T I needed to keep the scientific content accurate while moving the complete initiative to an agreed delivery date.

A I defined the technical scope with color scientists and marketing partners, converted the work into milestones and owners, reviewed claims and demonstrations against the data, surfaced dependencies early, and coordinated final readiness checks.

R The initiative was delivered on time and in full with aligned technical evidence, customer messaging, and operational support.

ppg-01

Improved color-matching classification accuracy using numerical integration methods.

S Automotive-refinish color matching depended on classifying candidate formulas from spectral and colorimetric data, but the existing calculation lost useful information when it approximated continuous response curves.

T I needed to improve classification accuracy without making the production workflow impractically slow.

A I reviewed the mathematical assumptions, implemented numerical integration over the measured curves, compared alternative integration schemes on representative panels, and validated the classifications against known matches and expert review.

R The revised method separated close candidates more reliably and gave the matching team a stronger quantitative basis for selecting formulas.

ppg-02

Built cross-instrument correlation software in VB.NET and JMP to improve measurement consistency and support statistical risk analysis.

S Spectrophotometers at different laboratories and customer locations could report small but operationally important differences for the same color sample.

T I needed to quantify those differences and make measurements more comparable across instruments.

A I built a VB.NET workflow to ingest and align instrument readings, used JMP to model bias and variation by device and color region, generated correlation adjustments, and summarized residual risk for technical stakeholders.

R Teams could identify out-of-family instruments, apply a consistent correlation approach, and make color decisions with a clearer understanding of measurement uncertainty.

ppg-09

Cowrote color workflows and best practices for internal operations and external customers, supported the change-management efforts required for adoption.

S New color tools and analytical methods would not improve results if internal teams and customers continued to follow inconsistent measurement and matching practices.

T I needed to turn technical knowledge into usable workflows and support the behavior change required for adoption.

A I cowrote step-by-step practices with operational experts, added decision points and troubleshooting examples, tested the guidance with representative users, supported training and rollout communications, and refined the material from field feedback.

R Users received a clearer, repeatable way to apply the tools, and adoption discussions focused on observable practices rather than abstract scientific concepts.

ppg-13

Informed marketing strategy and communications by connecting customer needs with technical capabilities and broad operational insight.

S Marketing needed to describe color capabilities in terms that mattered to refinish customers without overstating what the technology or operating network could deliver.

T I needed to connect customer problems, scientific capabilities, and operational realities into credible positioning.

A I gathered questions from customer-facing and regional teams, mapped them to product and process capabilities, reviewed proposed claims against technical evidence, and supplied examples that explained the practical value and boundaries of the solution.

R Marketing communications were better aligned with customer needs and technical reality, giving commercial teams a more credible story and clearer guidance on where the offering fit.

ppg-04

Wrote a Visual C++ and VB.NET DLL wrapper for a loaned spectroscopic instrument and developed formulation methods for an automated color-matching facility.

S PPG was evaluating a loaned spectroscopic instrument for an automated color-matching facility, but its native interface did not integrate with the existing VB.NET formulation environment.

T I needed to establish reliable software control of the instrument and turn its measurements into usable formulation inputs.

A I studied the vendor interface, wrote a Visual C++ DLL wrapper, exposed a stable VB.NET contract, handled connection and error states, and developed and tested formulation methods using controlled samples.

R The team could exercise the instrument through the planned automation stack and evaluate an end-to-end measurement-to-formulation workflow before making a larger equipment commitment.

ppg-03

Debugged a VB.NET and FORTRAN optimization routine for a numerical radiative-transfer solution.

S A radiative-transfer optimizer combined a VB.NET application with a legacy FORTRAN numerical routine, and some inputs produced unstable or nonconvergent formulation results.

T I needed to determine whether the problem came from interlanguage data exchange, numerical assumptions, or the optimization logic itself.

A I traced representative cases across the VB.NET/FORTRAN boundary, verified array dimensions and parameter units, instrumented intermediate values, reproduced edge conditions, and corrected the faulty convergence and input-handling behavior.

R The solver produced stable results across a broader operating range and became easier for the team to diagnose when unusual samples were encountered.